

Deep Learning : Lecture 6

Regularization: Bias Variance Tradeoff, l_2 regularization, Early stopping, Dataset augmentation, Parameter sharing and tying, Injecting noise at input, Ensemble methods, Dropout

Acknowledgements

- Chapter 7, Deep Learning book
- Ali Ghodsi's Video Lectures on Regularization^a
- Dropout: A Simple Way to Prevent Neural Networks from Overfitting^b
- CS6910: Deep Learning Course by Prof. Mitesh M. Khapra, IIT Madras, India

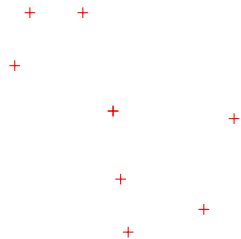
^aLecture 2.1 and Lecture 2.2

^bDropout

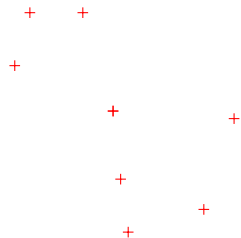
Module 8.1 : Bias and Variance

We will begin with a quick overview of bias, variance and the trade-off between them.

- Let us consider the problem of fitting a curve through a given set of points

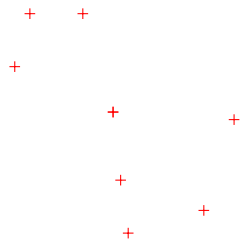


The points were drawn from a sinusoidal function (the true $f(x)$)



The points were drawn from a sinusoidal function (the true $f(x)$)

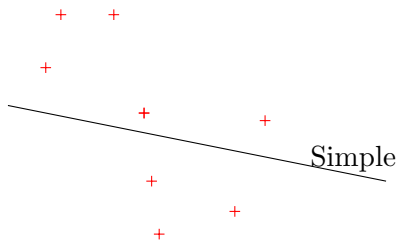
- Let us consider the problem of fitting a curve through a given set of points
- We consider two models :



The points were drawn from a sinusoidal function (the true $f(x)$)

- Let us consider the problem of fitting a curve through a given set of points
- We consider two models :

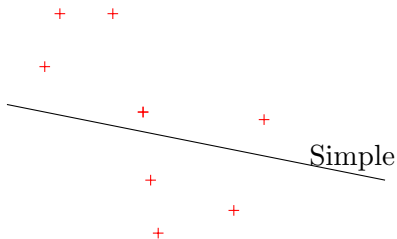
$$\begin{matrix} \textit{Simple} \\ \textit{(degree:1)} \end{matrix} \quad y = \hat{f}(x) = w_1x + w_0$$



The points were drawn from a sinusoidal function (the true $f(x)$)

- Let us consider the problem of fitting a curve through a given set of points
- We consider two models :

$$\begin{matrix} \textit{Simple} \\ \textit{(degree:1)} \end{matrix} \quad y = \hat{f}(x) = w_1x + w_0$$

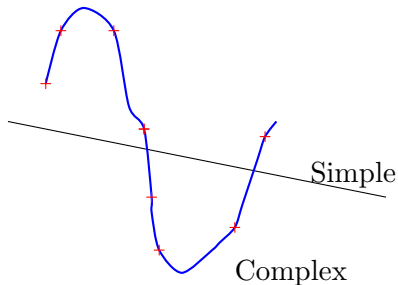


The points were drawn from a sinusoidal function (the true $f(x)$)

- Let us consider the problem of fitting a curve through a given set of points
- We consider two models :

$$\begin{array}{l} \textit{Simple} \\ \textit{(degree:1)} \end{array} \quad y = \hat{f}(x) = w_1x + w_0$$

$$\begin{array}{l} \textit{Complex} \\ \textit{(degree:25)} \end{array} \quad y = \hat{f}(x) = \sum_{i=1}^{25} w_i x^i + w_0$$

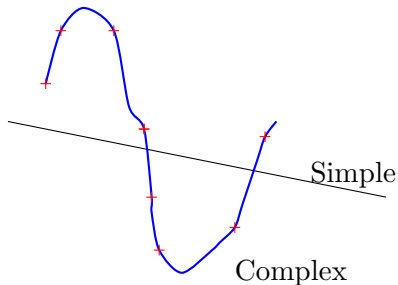


The points were drawn from a sinusoidal function (the true $f(x)$)

- Let us consider the problem of fitting a curve through a given set of points
- We consider two models :

$$\begin{array}{l} \text{Simple} \\ (\text{degree:1}) \end{array} \quad y = \hat{f}(x) = w_1x + w_0$$

$$\begin{array}{l} \text{Complex} \\ (\text{degree:25}) \end{array} \quad y = \hat{f}(x) = \sum_{i=1}^{25} w_i x^i + w_0$$



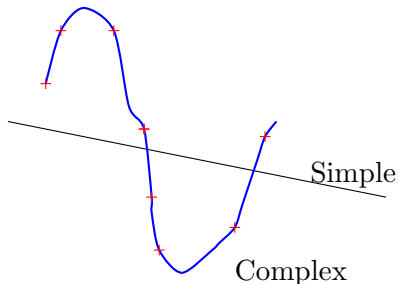
The points were drawn from a sinusoidal function (the true $f(x)$)

- Let us consider the problem of fitting a curve through a given set of points
- We consider two models :

$$\begin{array}{l} \text{Simple} \\ (\text{degree:1}) \end{array} \quad y = \hat{f}(x) = w_1x + w_0$$

$$\begin{array}{l} \text{Complex} \\ (\text{degree:25}) \end{array} \quad y = \hat{f}(x) = \sum_{i=1}^{25} w_i x^i + w_0$$

- Note that in both cases we are making an assumption about how y is related to x . We have no idea about the true relation $f(x)$



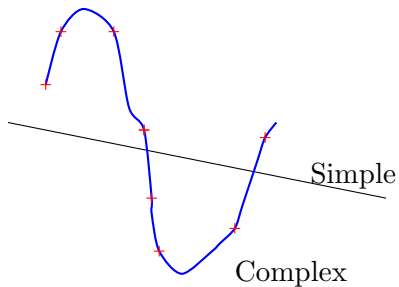
The points were drawn from a sinusoidal function (the true $f(x)$)

- Let us consider the problem of fitting a curve through a given set of points
- We consider two models :

$$\begin{matrix} \text{Simple} \\ (\text{degree:1}) \end{matrix} \quad y = \hat{f}(x) = w_1x + w_0$$

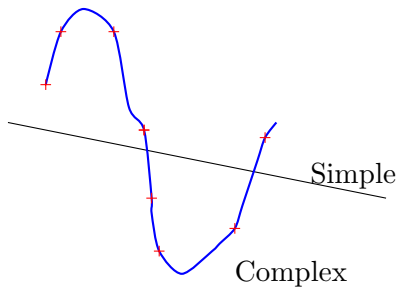
$$\begin{matrix} \text{Complex} \\ (\text{degree:25}) \end{matrix} \quad y = \hat{f}(x) = \sum_{i=1}^{25} w_i x^i + w_0$$

- Note that in both cases we are making an assumption about how y is related to x . We have no idea about the true relation $f(x)$
- The training data consists of 100 points



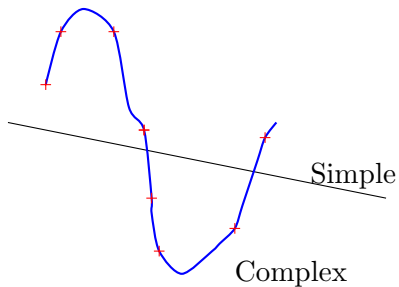
The points were drawn from a sinusoidal function (the true $f(x)$)

- We sample 25 points from the training data and train a simple and a complex model



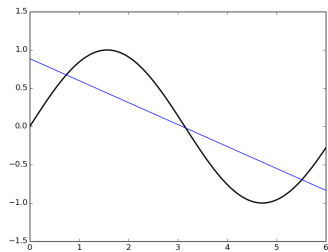
The points were drawn from a sinusoidal function (the true $f(x)$)

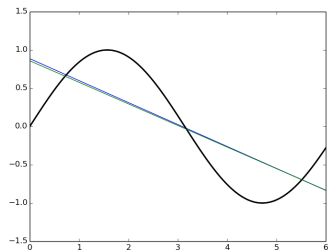
- We sample 25 points from the training data and train a simple and a complex model
- We repeat the process ' k ' times to train multiple models (each model sees a different sample of the training data)

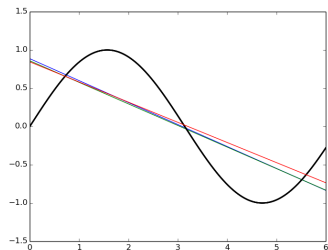


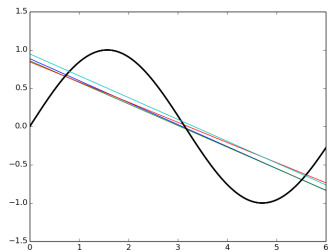
The points were drawn from a sinusoidal function (the true $f(x)$)

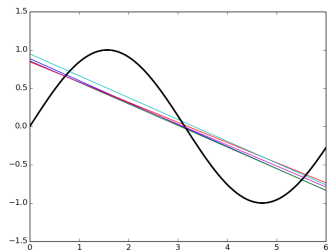
- We sample 25 points from the training data and train a simple and a complex model
- We repeat the process ' k ' times to train multiple models (each model sees a different sample of the training data)
- We make a few observations from these plots

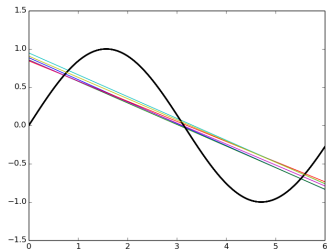


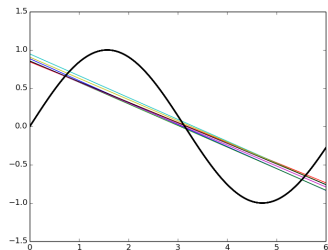


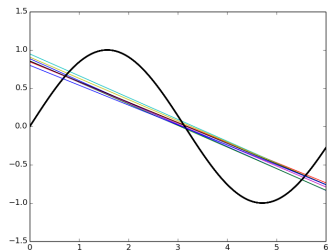


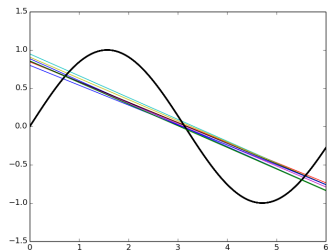


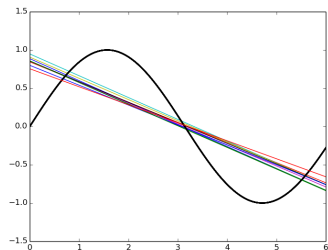


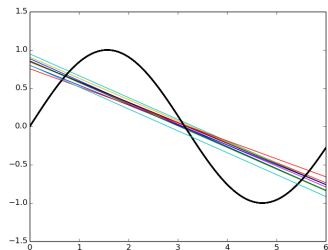


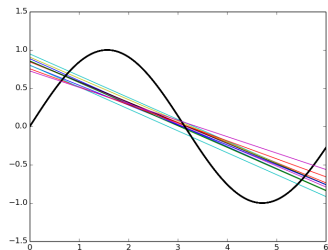


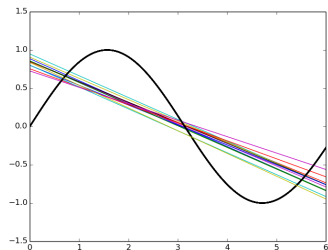


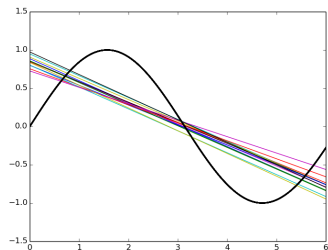


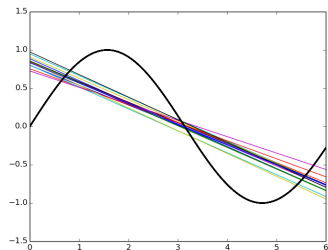


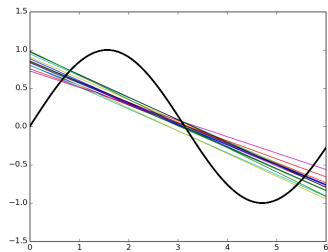


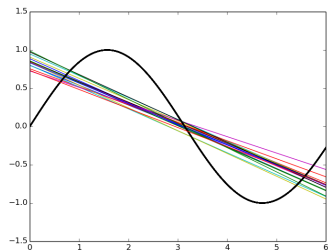


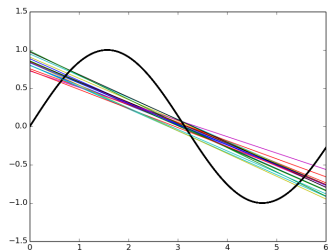


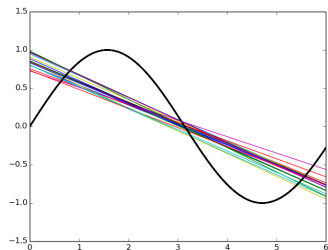


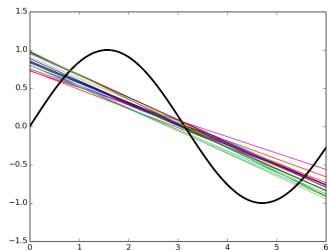


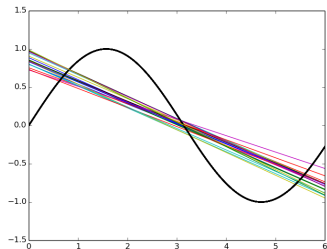




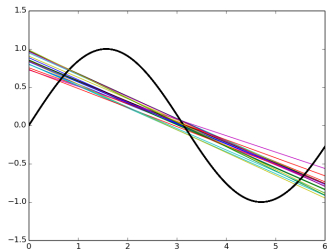




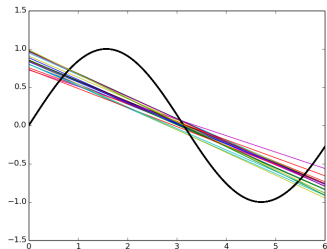




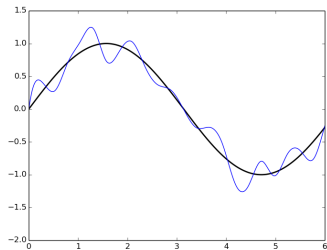
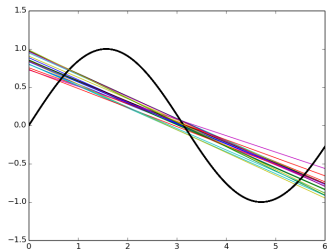
- Simple models trained on different samples of the data do not differ much from each other



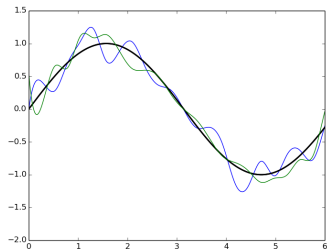
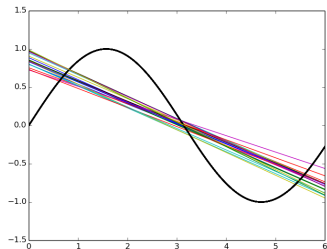
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



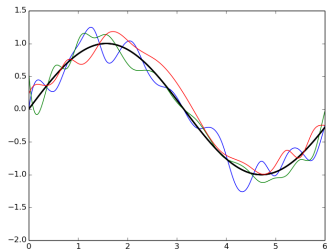
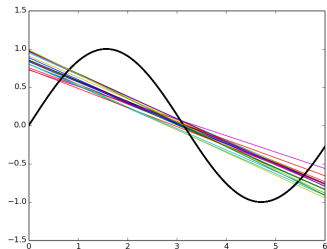
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



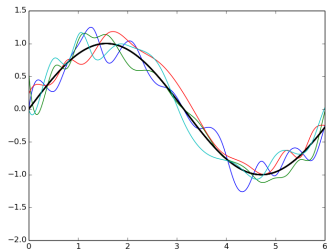
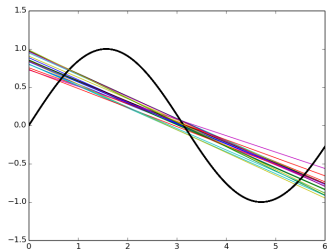
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



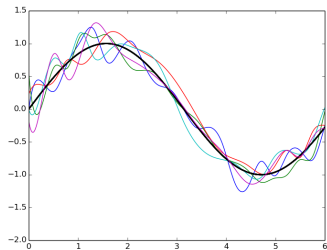
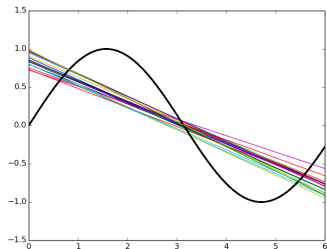
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



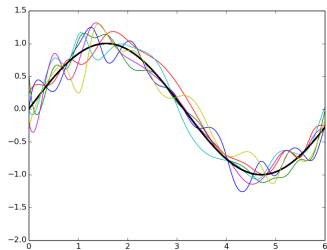
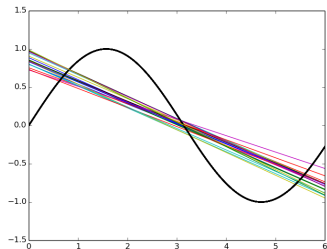
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



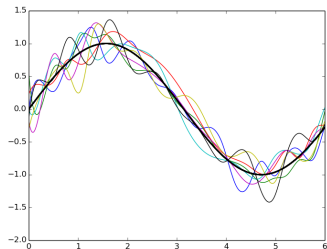
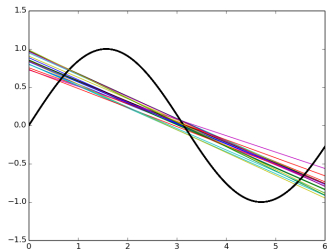
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



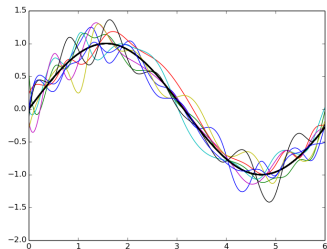
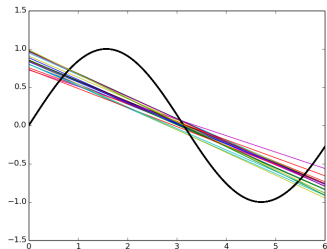
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



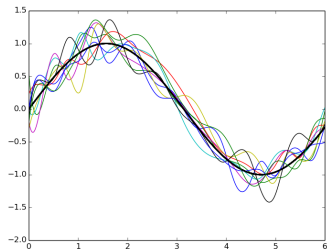
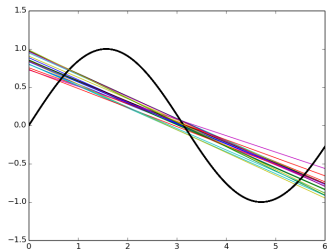
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



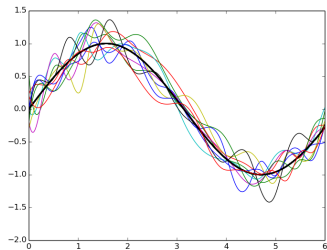
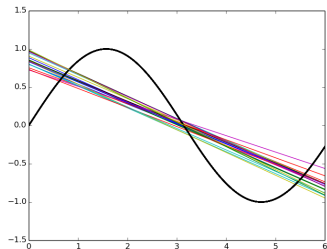
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



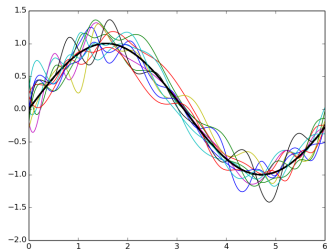
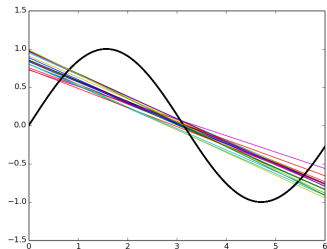
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



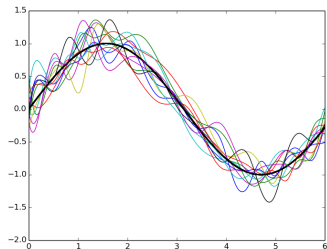
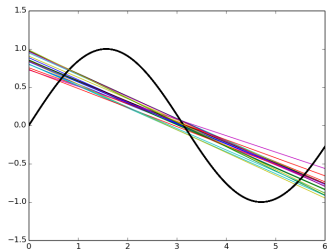
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



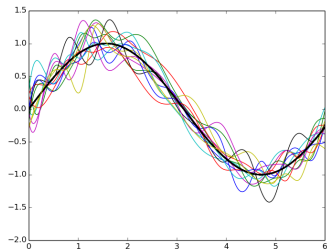
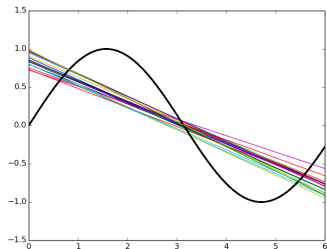
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



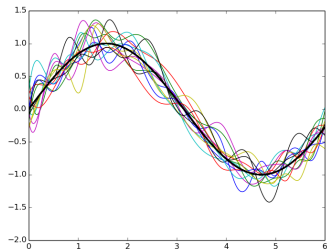
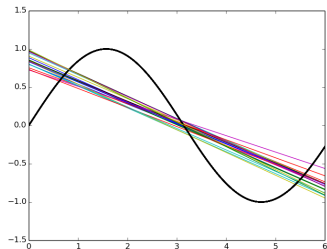
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



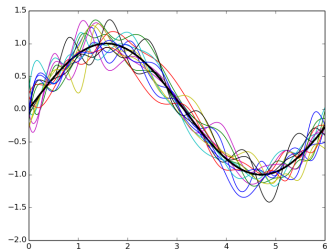
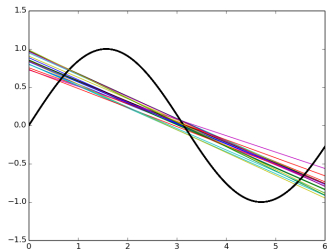
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



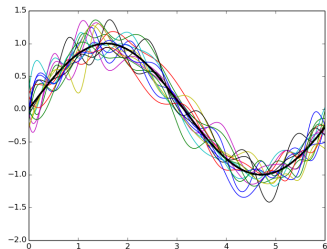
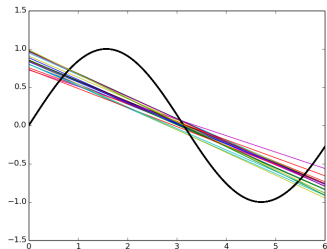
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



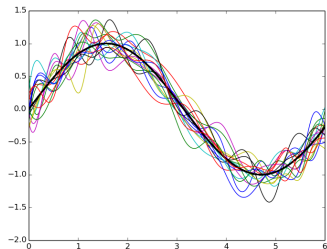
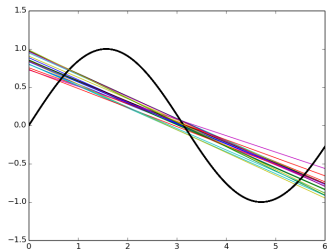
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



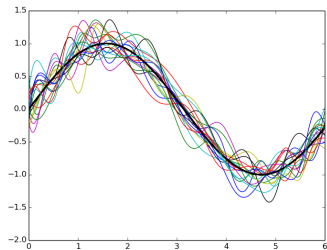
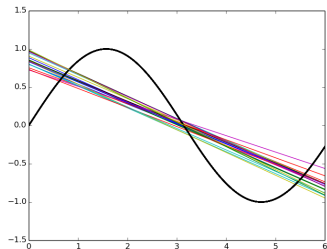
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



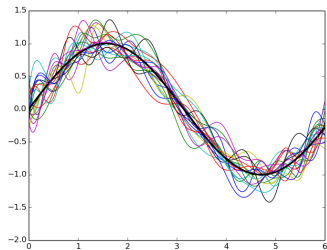
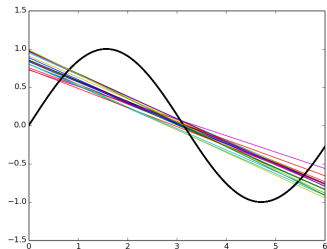
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



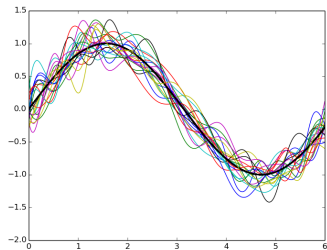
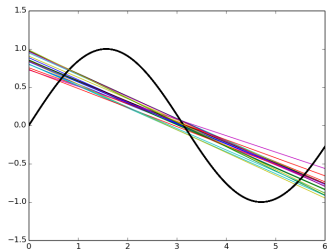
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



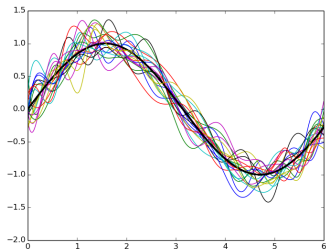
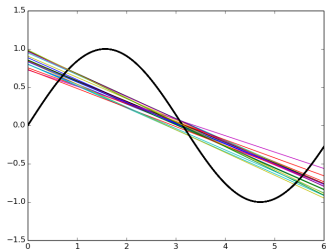
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



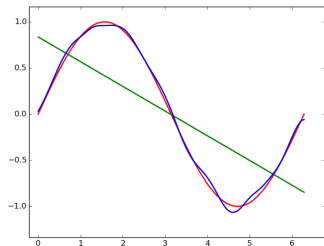
- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)



- Simple models trained on different samples of the data do not differ much from each other
- However they are very far from the true sinusoidal curve (under fitting)
- On the other hand, complex models trained on different samples of the data are very different from each other (high variance)

- Let $f(x)$ be the true model (sinusoidal in this case) and $\hat{f}(x)$ be our estimate of the model (simple or complex, in this case) then,

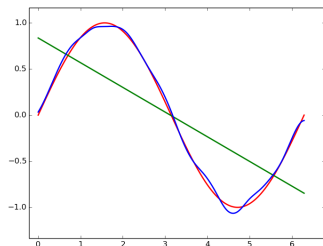
$$\text{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$$



Green Line: Average value of $\hat{f}(x)$
for the simple model

Blue Curve: Average value of $\hat{f}(x)$
for the complex model

Red Curve: True model ($f(x)$)



Green Line: Average value of $\hat{f}(x)$
for the simple model

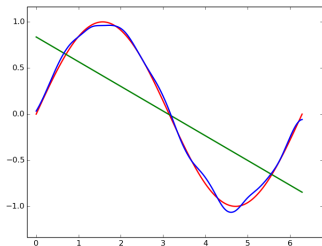
Blue Curve: Average value of $\hat{f}(x)$
for the complex model

Red Curve: True model ($f(x)$)

- Let $f(x)$ be the true model (sinusoidal in this case) and $\hat{f}(x)$ be our estimate of the model (simple or complex, in this case) then,

$$\text{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$$

- $E[\hat{f}(x)]$ is the average (or expected) value of the model



Green Line: Average value of $\hat{f}(x)$
for the simple model

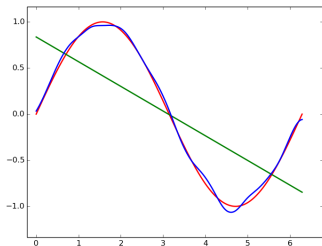
Blue Curve: Average value of $\hat{f}(x)$
for the complex model

Red Curve: True model ($f(x)$)

- Let $f(x)$ be the true model (sinusoidal in this case) and $\hat{f}(x)$ be our estimate of the model (simple or complex, in this case) then,

$$\text{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$$

- $E[\hat{f}(x)]$ is the average (or expected) value of the model
- We can see that for the simple model the average value (green line) is very far from the true value $f(x)$ (sinusoidal function)



Green Line: Average value of $\hat{f}(x)$ for the simple model

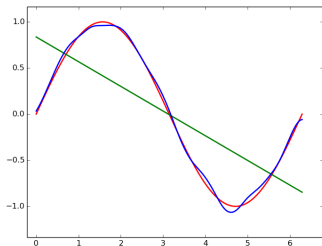
Blue Curve: Average value of $\hat{f}(x)$ for the complex model

Red Curve: True model ($f(x)$)

- Let $f(x)$ be the true model (sinusoidal in this case) and $\hat{f}(x)$ be our estimate of the model (simple or complex, in this case) then,

$$\text{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$$

- $E[\hat{f}(x)]$ is the average (or expected) value of the model
- We can see that for the simple model the average value (green line) is very far from the true value $f(x)$ (sinusoidal function)
- Mathematically, this means that the simple model has a high bias



Green Line: Average value of $\hat{f}(x)$ for the simple model

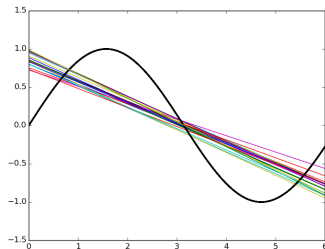
Blue Curve: Average value of $\hat{f}(x)$ for the complex model

Red Curve: True model ($f(x)$)

- Let $f(x)$ be the true model (sinusoidal in this case) and $\hat{f}(x)$ be our estimate of the model (simple or complex, in this case) then,

$$\text{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$$

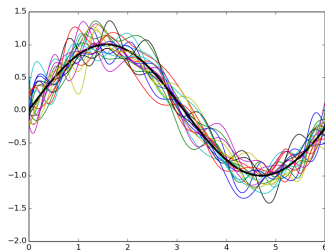
- $E[\hat{f}(x)]$ is the average (or expected) value of the model
- We can see that for the simple model the average value (green line) is very far from the true value $f(x)$ (sinusoidal function)
- Mathematically, this means that the simple model has a high bias
- On the other hand, the complex model has a low bias

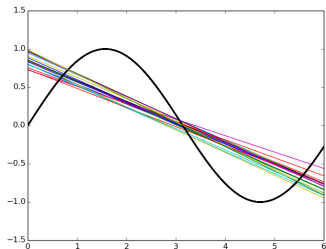


- We now define,

$$\text{Variance } (\hat{f}(x)) = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$$

(Standard definition from statistics)



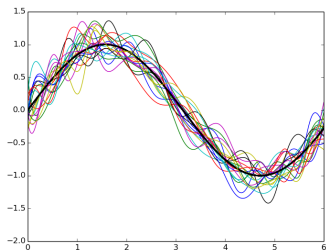


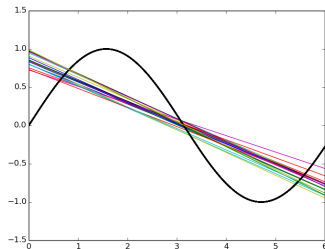
- We now define,

$$\text{Variance } (\hat{f}(x)) = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$$

(Standard definition from statistics)

- Roughly speaking it tells us how much the different $\hat{f}(x)$'s (trained on different samples of the data) differ from each other

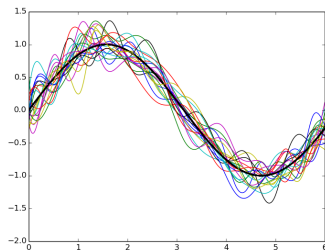




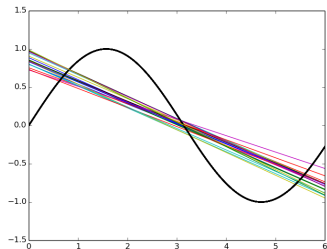
- We now define,

$$\text{Variance } (\hat{f}(x)) = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$$

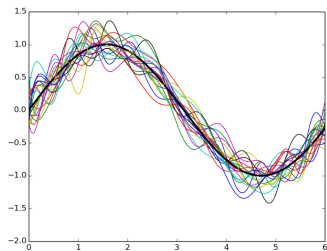
(Standard definition from statistics)

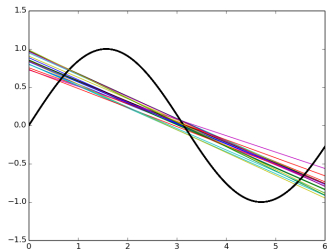


- Roughly speaking it tells us how much the different $\hat{f}(x)$'s (trained on different samples of the data) differ from each other
- It is clear that the simple model has a low variance whereas the complex model has a high variance

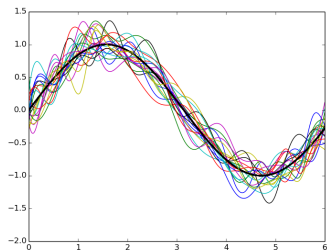


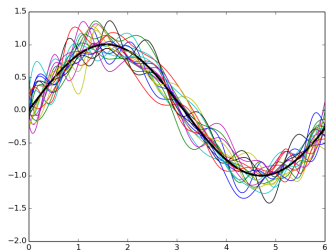
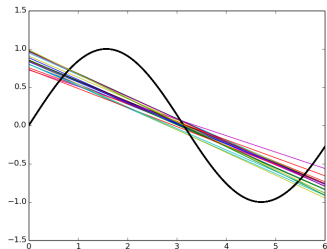
- In summary (informally)



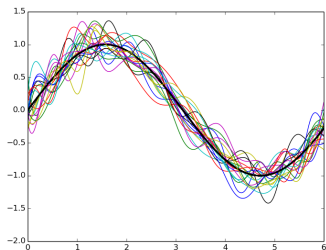
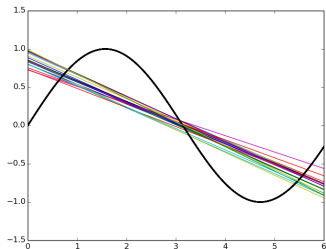


- In summary (informally)
- Simple model: high bias, low variance

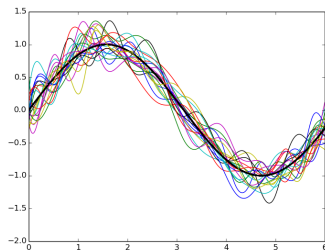
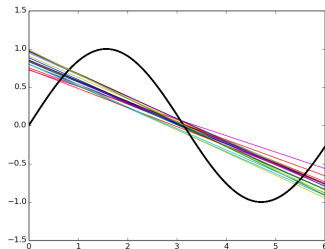




- In summary (informally)
- Simple model: high bias, low variance
- Complex model: low bias, high variance



- In summary (informally)
- Simple model: high bias, low variance
- Complex model: low bias, high variance
- There is always a trade-off between the bias and variance



- In summary (informally)
- Simple model: high bias, low variance
- Complex model: low bias, high variance
- There is always a trade-off between the bias and variance
- Both bias and variance contribute to the mean square error. Let us see how

Module 8.2 : Train error vs Test error

- Consider a new point (x, y) which was not seen during training

- Consider a new point (x, y) which was not seen during training
- If we use the model $\hat{f}(x)$ to predict the value of y then the mean square error is given by

$$E[(y - \hat{f}(x))^2]$$

(average square error in predicting y for many such unseen points)

- We can show that

$$\begin{aligned} E[(y - \hat{f}(x))^2] &= Bias^2 \\ &+ Variance \\ &+ \sigma^2 \text{ (irreducible error)} \end{aligned}$$

- Consider a new point (x, y) which was not seen during training
- If we use the model $\hat{f}(x)$ to predict the value of y then the mean square error is given by

$$E[(y - \hat{f}(x))^2]$$

(average square error in predicting y for many such unseen points)

- We can show that

$$\begin{aligned} E[(y - \hat{f}(x))^2] &= Bias^2 \\ &+ Variance \\ &+ \sigma^2 \text{ (irreducible error)} \end{aligned}$$

- [See proof here](#)

- Consider a new point (x, y) which was not seen during training
- If we use the model $\hat{f}(x)$ to predict the value of y then the mean square error is given by

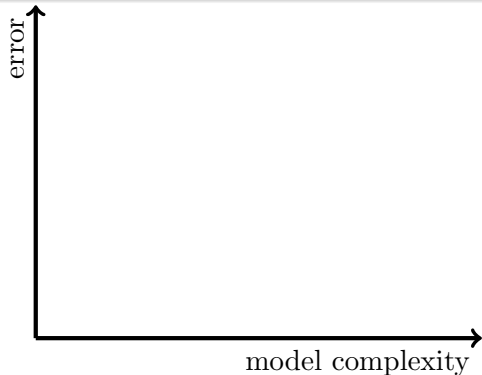
$$E[(y - \hat{f}(x))^2]$$

(average square error in predicting y for many such unseen points)

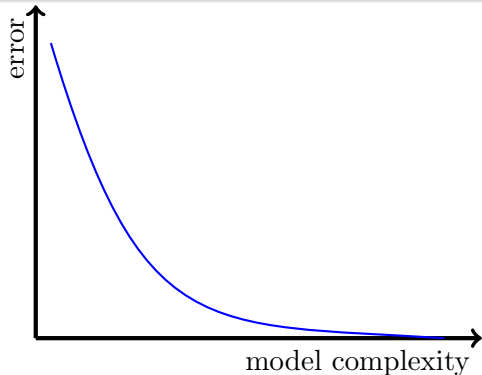
- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$

- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training

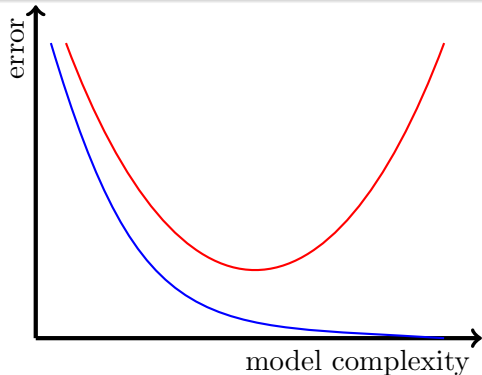
- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training
- This gives rise to the following two entities of interest:
train_{err} (say, mean square error)
test_{err} (say, mean square error)



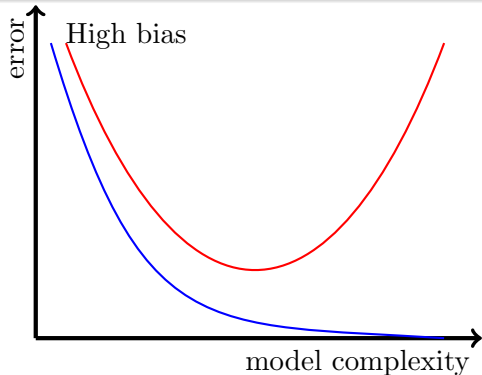
- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training
- This gives rise to the following two entities of interest:
 - $train_{err}$ (say, mean square error)
 - $test_{err}$ (say, mean square error)
- Typically these errors exhibit the trend shown in the adjacent figure



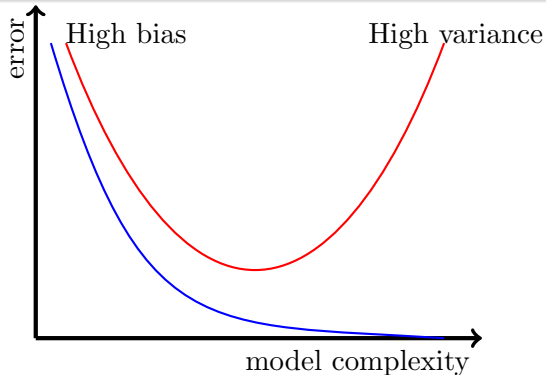
- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training
- This gives rise to the following two entities of interest:
train_{err} (say, mean square error)
test_{err} (say, mean square error)
- Typically these errors exhibit the trend shown in the adjacent figure



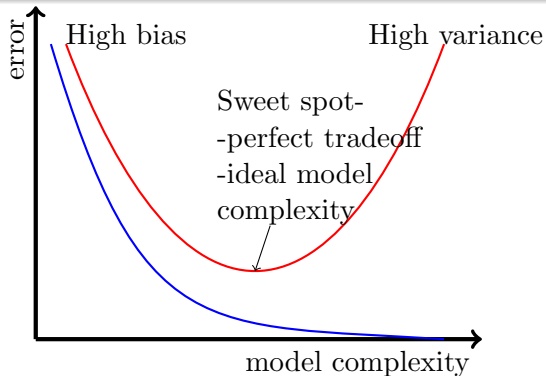
- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training
- This gives rise to the following two entities of interest:
train_{err} (say, mean square error)
test_{err} (say, mean square error)
- Typically these errors exhibit the trend shown in the adjacent figure



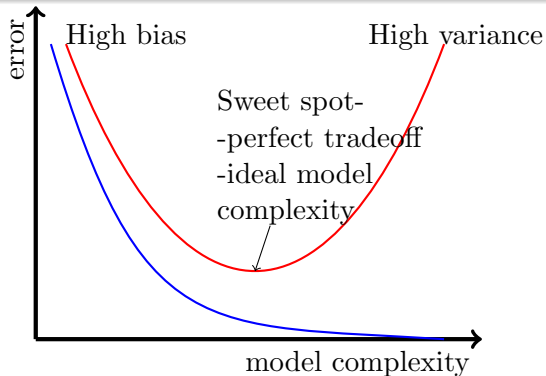
- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training
- This gives rise to the following two entities of interest:
train_{err} (say, mean square error)
test_{err} (say, mean square error)
- Typically these errors exhibit the trend shown in the adjacent figure



- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training
- This gives rise to the following two entities of interest:
train_{err} (say, mean square error)
test_{err} (say, mean square error)
- Typically these errors exhibit the trend shown in the adjacent figure



- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training
- This gives rise to the following two entities of interest:
train_{err} (say, mean square error)
test_{err} (say, mean square error)
- Typically these errors exhibit the trend shown in the adjacent figure



$$\begin{aligned}
 E[(y - \hat{f}(x))^2] &= Bias^2 \\
 &+ Variance \\
 &+ \sigma^2 \text{ (irreducible error)}
 \end{aligned}$$

- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set $\{(x_i, y_i)\}_{i=1}^n$
- However, at test time we are interested in evaluating the model on a validation (unseen) set which was not used for training
- This gives rise to the following two entities of interest:
train_{err} (say, mean square error)
test_{err} (say, mean square error)
- Typically these errors exhibit the trend shown in the adjacent figure

Intuitions developed so far

- Let there be n training points and m test (validation) points

$$train_{err} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

$$test_{err} = \frac{1}{m} \sum_{i=n+1}^{n+m} (y_i - \hat{f}(x_i))^2$$

Intuitions developed so far

- Let there be n training points and m test (validation) points

$$train_{err} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

$$test_{err} = \frac{1}{m} \sum_{i=n+1}^{n+m} (y_i - \hat{f}(x_i))^2$$

- As the model complexity increases $train_{err}$ becomes overly optimistic and gives us a wrong picture of how close $\hat{f}$ is to f

Intuitions developed so far

- Let there be n training points and m test (validation) points

$$train_{err} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

$$test_{err} = \frac{1}{m} \sum_{i=n+1}^{n+m} (y_i - \hat{f}(x_i))^2$$

- As the model complexity increases $train_{err}$ becomes overly optimistic and gives us a wrong picture of how close $\hat{f}$ is to f
- The validation error gives the real picture of how close $\hat{f}$ is to f

Intuitions developed so far

- Let there be n training points and m test (validation) points

$$train_{err} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

$$test_{err} = \frac{1}{m} \sum_{i=n+1}^{n+m} (y_i - \hat{f}(x_i))^2$$

- As the model complexity increases $train_{err}$ becomes overly optimistic and gives us a wrong picture of how close $\hat{f}$ is to f
- The validation error gives the real picture of how close $\hat{f}$ is to f
- We will concretize this intuition mathematically now and eventually show how to account for the optimism in the training error

- Let $D = \{x_i, y_i\}_{i=1}^{m+n}$, then for any point (x, y) we have,

$$y_i = f(x_i) + \varepsilon_i$$

- Let $D = \{x_i, y_i\}_{i=1}^{m+n}$, then for any point (x, y) we have,

$$y_i = f(x_i) + \varepsilon_i$$

- which means that y_i is related to x_i by some true function f but there is also some noise ε in the relation

- Let $D = \{x_i, y_i\}_{i=1}^{m+n}$, then for any point (x, y) we have,

$$y_i = f(x_i) + \varepsilon_i$$

- which means that y_i is related to x_i by some true function f but there is also some noise ε in the relation
- For simplicity, we assume

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

- Let $D = \{x_i, y_i\}_{i=1}^{m+n}$, then for any point (x, y) we have,

$$y_i = f(x_i) + \varepsilon_i$$

- which means that y_i is related to x_i by some true function f but there is also some noise ε in the relation
- For simplicity, we assume

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

and of course we do not know f

- Let $D = \{x_i, y_i\}_{i=1}^{m+n}$, then for any point (x, y) we have,

$$y_i = f(x_i) + \varepsilon_i$$

- which means that y_i is related to x_i by some true function f but there is also some noise ε in the relation
- For simplicity, we assume

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

and of course we do not know f

- Further we use $\hat{f}$ to approximate f and estimate the parameters using $T \subset D$ such that

$$y_i = \hat{f}(x_i)$$

- Let $D = \{x_i, y_i\}_{i=1}^{m+n}$, then for any point (x, y) we have,

$$y_i = f(x_i) + \varepsilon_i$$

- which means that y_i is related to x_i by some true function f but there is also some noise ε in the relation
- For simplicity, we assume

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

and of course we do not know f

- Further we use $\hat{f}$ to approximate f and estimate the parameters using $T \subset D$ such that

$$y_i = \hat{f}(x_i)$$

- We are interested in knowing

$$E[(\hat{f}(x_i) - f(x_i))^2]$$

- Let $D = \{x_i, y_i\}_{i=1}^{m+n}$, then for any point (x, y) we have,

$$y_i = f(x_i) + \varepsilon_i$$

- which means that y_i is related to x_i by some true function f but there is also some noise ε in the relation

- For simplicity, we assume

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

and of course we do not know f

- Further we use $\hat{f}$ to approximate f and estimate the parameters using $T \subset D$ such that

$$y_i = \hat{f}(x_i)$$

- We are interested in knowing

$$E[(\hat{f}(x_i) - f(x_i))^2]$$

but we cannot estimate this directly because we do not know f

- Let $D = \{x_i, y_i\}_{i=1}^{m+n}$, then for any point (x, y) we have,

$$y_i = f(x_i) + \varepsilon_i$$

- which means that y_i is related to x_i by some true function f but there is also some noise ε in the relation

- For simplicity, we assume

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

and of course we do not know f

- Further we use $\hat{f}$ to approximate f and estimate the parameters using $T \subset D$ such that

$$\hat{y}_i = \hat{f}(x_i)$$

- We are interested in knowing

$$E[(\hat{f}(x_i) - f(x_i))^2]$$

but we cannot estimate this directly because we do not know f

- We will see how to estimate this empirically using the observation y_i & prediction $\hat{y}_i$

$$E[(\hat{y}_i - y_i)^2]$$

$$E[(\hat{y}_i - y_i)^2] = E[(\hat{f}(x_i) - f(x_i) - \varepsilon_i)^2] \quad (y_i = f(x_i) + \varepsilon_i)$$

$$\begin{aligned} E[(\hat{y}_i - y_i)^2] &= E[(\hat{f}(x_i) - f(x_i) - \varepsilon_i)^2] \quad (y_i = f(x_i) + \varepsilon_i) \\ &= E[(\hat{f}(x_i) - f(x_i))^2 - 2\varepsilon_i(\hat{f}(x_i) - f(x_i)) + \varepsilon_i^2] \end{aligned}$$

$$E[(\hat{y}_i - y_i)^2] = E[(\hat{f}(x_i) - f(x_i) - \varepsilon_i)^2] \quad (y_i = f(x_i) + \varepsilon_i)$$

$$= E[(\hat{f}(x_i) - f(x_i))^2 - 2\varepsilon_i(\hat{f}(x_i) - f(x_i)) + \varepsilon_i^2]$$

$$= E[(\hat{f}(x_i) - f(x_i))^2] - 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))] + E[\varepsilon_i^2]$$

$$E[(\hat{y}_i - y_i)^2] = E[(\hat{f}(x_i) - f(x_i) - \varepsilon_i)^2] \quad (y_i = f(x_i) + \varepsilon_i)$$

$$= E[(\hat{f}(x_i) - f(x_i))^2 - 2\varepsilon_i(\hat{f}(x_i) - f(x_i)) + \varepsilon_i^2]$$

$$= E[(\hat{f}(x_i) - f(x_i))^2] - 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))] + E[\varepsilon_i^2]$$

$$\therefore E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

We will take a small detour to understand how to empirically estimate an Expectation and then return to our derivation

- Suppose we have observed the goals scored(z) in k matches as $z_1 = 2, z_2 = 1, z_3 = 0, \dots z_k = 2$

- Suppose we have observed the goals scored(z) in k matches as $z_1 = 2, z_2 = 1, z_3 = 0, \dots z_k = 2$
- Now we can empirically estimate $E[z]$ i.e. the expected number of goals scored as

$$E[z] = \frac{1}{k} \sum_{i=1}^k z_i$$

- Suppose we have observed the goals scored(z) in k matches as $z_1 = 2, z_2 = 1, z_3 = 0, \dots z_k = 2$
- Now we can empirically estimate $E[z]$ i.e. the expected number of goals scored as

$$E[z] = \frac{1}{k} \sum_{i=1}^k z_i$$

- Analogy with our derivation: We have a certain number of observations y_i & predictions $\hat{y}_i$ using which we can estimate

$$E[(\hat{y}_i - y_i)^2] =$$

- Suppose we have observed the goals scored(z) in k matches as $z_1 = 2, z_2 = 1, z_3 = 0, \dots z_k = 2$
- Now we can empirically estimate $E[z]$ i.e. the expected number of goals scored as

$$E[z] = \frac{1}{k} \sum_{i=1}^k z_i$$

- Analogy with our derivation: We have a certain number of observations y_i & predictions $\hat{y}_i$ using which we can estimate

$$E[(\hat{y}_i - y_i)^2] = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

... returning back to our derivation

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}}$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} -$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}}$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance } (\varepsilon_i, \hat{f}(x_i) - f(x_i))}$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}$$

$\therefore \text{covariance}(X, Y)$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}$$

$$\because \text{covariance}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}$$

$$\begin{aligned} \because \text{covariance}(X, Y) &= E[(X - \mu_X)(Y - \mu_Y)] \\ &= E[(X)(Y - \mu_Y)] \text{ (if } \mu_X = E[X] = 0) \end{aligned}$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}$$

$$\begin{aligned} \because \text{covariance}(X, Y) &= E[(X - \mu_X)(Y - \mu_Y)] \\ &= E[(X)(Y - \mu_Y)] (\text{if } \mu_X = E[X] = 0) \\ &= E[XY] - E[X\mu_Y] \end{aligned}$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}$$

$$\begin{aligned} \because \text{covariance}(X, Y) &= E[(X - \mu_X)(Y - \mu_Y)] \\ &= E[(X)(Y - \mu_Y)] \text{ (if } \mu_X = E[X] = 0) \\ &= E[XY] - E[X\mu_Y] = E[XY] - \mu_Y E[X] \end{aligned}$$

$$E[(\hat{f}(x_i) - f(x_i))^2] = E[(\hat{y}_i - y_i)^2] - E[\varepsilon_i^2] + 2E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$$

- We can empirically evaluate R.H.S using training observations or test observations

Case 1: Using test observations

$$\underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} = \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}$$

$$\begin{aligned} \because \text{covariance}(X, Y) &= E[(X - \mu_X)(Y - \mu_Y)] \\ &= E[(X)(Y - \mu_Y)] \text{ (if } \mu_X = E[X] = 0) \\ &= E[XY] - E[X\mu_Y] = E[XY] - \mu_Y E[X] = E[XY] \end{aligned}$$

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
= & \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
&= \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

- None of the test observations participated in the estimation of $\hat{f}(x)$ [the parameters of $\hat{f}(x)$ were estimated only using training data]

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
&= \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

- None of the test observations participated in the estimation of $\hat{f}(x)$ [the parameters of $\hat{f}(x)$ were estimated only using training data]

$$\therefore \varepsilon \perp (\hat{f}(x_i) - f(x_i))$$

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
&= \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

- None of the test observations participated in the estimation of $\hat{f}(x)$ [the parameters of $\hat{f}(x)$ were estimated only using training data]

$$\therefore \varepsilon \perp (\hat{f}(x_i) - f(x_i))$$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] = 0$$

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
&= \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

- None of the test observations participated in the estimation of $\hat{f}(x)$ [the parameters of $\hat{f}(x)$ were estimated only using training data]

$$\therefore \varepsilon \perp (\hat{f}(x_i) - f(x_i))$$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] = E[\varepsilon_i] \cdot E[\hat{f}(x_i) - f(x_i)]$$

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
&= \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

- None of the test observations participated in the estimation of $\hat{f}(x)$ [the parameters of $\hat{f}(x)$ were estimated only using training data]

$$\therefore \varepsilon \perp (\hat{f}(x_i) - f(x_i))$$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] = E[\varepsilon_i] \cdot E[\hat{f}(x_i) - f(x_i)] = 0 \cdot E[\hat{f}(x_i) - f(x_i)]$$

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
&= \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

- None of the test observations participated in the estimation of $\hat{f}(x)$ [the parameters of $\hat{f}(x)$ were estimated only using training data]

$$\therefore \varepsilon \perp (\hat{f}(x_i) - f(x_i))$$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] = E[\varepsilon_i] \cdot E[\hat{f}(x_i) - f(x_i)] = 0 \cdot E[\hat{f}(x_i) - f(x_i)] = 0$$

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
&= \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

- None of the test observations participated in the estimation of $\hat{f}(x)$ [the parameters of $\hat{f}(x)$ were estimated only using training data]

$$\therefore \varepsilon \perp (\hat{f}(x_i) - f(x_i))$$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] = E[\varepsilon_i] \cdot E[\hat{f}(x_i) - f(x_i)] = 0 \cdot E[\hat{f}(x_i) - f(x_i)] = 0$$

$$\therefore \text{true error} = \text{empirical test error} + \text{small constant}$$

$$\begin{aligned}
& \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\
&= \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))}
\end{aligned}$$

- None of the test observations participated in the estimation of $\hat{f}(x)$ [the parameters of $\hat{f}(x)$ were estimated only using training data]

$$\therefore \varepsilon \perp (\hat{f}(x_i) - f(x_i))$$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] = E[\varepsilon_i] \cdot E[\hat{f}(x_i) - f(x_i)] = 0 \cdot E[\hat{f}(x_i) - f(x_i)] = 0$$

$$\therefore \text{true error} = \text{empirical test error} + \text{small constant}$$

- Hence, we should always use a validation set (independent of the training set) to estimate the error

Case 2: Using training observations

$$\begin{aligned} & \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\ &= \underbrace{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{n} \sum_{i=1}^n \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))} \end{aligned}$$

Case 2: Using training observations

$$\begin{aligned} & \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\ &= \underbrace{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{n} \sum_{i=1}^n \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))} \end{aligned}$$

Now, $\varepsilon \not\perp \hat{f}(x)$ because ε was used for estimating the parameters of $\hat{f}(x)$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))]$$

Case 2: Using training observations

$$\begin{aligned} & \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\ &= \underbrace{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{n} \sum_{i=1}^n \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))} \end{aligned}$$

Now, $\varepsilon \not\perp \hat{f}(x)$ because ε was used for estimating the parameters of $\hat{f}(x)$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))]$$

Case 2: Using training observations

$$\begin{aligned} & \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\ &= \underbrace{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{n} \sum_{i=1}^n \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))} \end{aligned}$$

Now, $\varepsilon \not\perp \hat{f}(x)$ because ε was used for estimating the parameters of $\hat{f}(x)$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] \neq E[\varepsilon_i] \cdot E[\hat{f}(x_i) - f(x_i)]$$

Case 2: Using training observations

$$\begin{aligned} & \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\ &= \underbrace{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{n} \sum_{i=1}^n \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))} \end{aligned}$$

Now, $\varepsilon \not\perp \hat{f}(x)$ because ε was used for estimating the parameters of $\hat{f}(x)$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] \neq E[\varepsilon_i] \cdot E[\hat{f}(x_i) - f(x_i)] \neq 0$$

Hence, the empirical train error is smaller than the true error and does not give a true picture of the error

Case 2: Using training observations

$$\begin{aligned} & \underbrace{E[(\hat{f}(x_i) - f(x_i))^2]}_{\text{true error}} \\ &= \underbrace{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}_{\text{empirical estimation of error}} - \underbrace{\frac{1}{n} \sum_{i=1}^n \varepsilon_i^2}_{\text{small constant}} + 2 \underbrace{E[\varepsilon_i(\hat{f}(x_i) - f(x_i))]}_{= \text{covariance}(\varepsilon_i, \hat{f}(x_i) - f(x_i))} \end{aligned}$$

Now, $\varepsilon \not\perp \hat{f}(x)$ because ε was used for estimating the parameters of $\hat{f}(x)$

$$\therefore E[\varepsilon_i \cdot (\hat{f}(x_i) - f(x_i))] \neq E[\varepsilon_i] \cdot E[\hat{f}(x_i) - f(x_i)] \neq 0$$

Hence, the empirical train error is smaller than the true error and does not give a true picture of the error

But how is this related to model complexity? Let us see

Module 8.3 : True error and Model complexity

Using Stein's Lemma (and some trickery) we can show that

$$\frac{1}{n} \sum_{i=1}^n \varepsilon_i (\hat{f}(x_i) - f(x_i)) = \frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$$

Using Stein's Lemma (and some trickery) we can show that

$$\frac{1}{n} \sum_{i=1}^n \varepsilon_i (\hat{f}(x_i) - f(x_i)) = \frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$$

- When will $\frac{\partial \hat{f}(x_i)}{\partial y_i}$ be high? When a small change in the observation causes a large change in the estimation($\hat{f}$)

Using Stein's Lemma (and some trickery) we can show that

$$\frac{1}{n} \sum_{i=1}^n \varepsilon_i(\hat{f}(x_i) - f(x_i)) = \frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$$

- When will $\frac{\partial \hat{f}(x_i)}{\partial y_i}$ be high? When a small change in the observation causes a large change in the estimation($\hat{f}$)
- Can you link this to model complexity?

Using Stein's Lemma (and some trickery) we can show that

$$\frac{1}{n} \sum_{i=1}^n \varepsilon_i (\hat{f}(x_i) - f(x_i)) = \frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$$

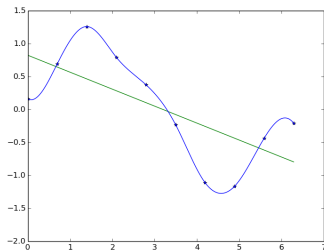
- When will $\frac{\partial \hat{f}(x_i)}{\partial y_i}$ be high? When a small change in the observation causes a large change in the estimation($\hat{f}$)
- Can you link this to model complexity?
- Yes, indeed a complex model will be more sensitive to changes in observations whereas a simple model will be less sensitive to changes in observations

Using Stein's Lemma (and some trickery) we can show that

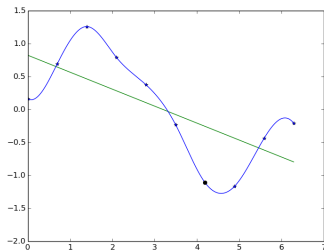
$$\frac{1}{n} \sum_{i=1}^n \varepsilon_i(\hat{f}(x_i) - f(x_i)) = \frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$$

- When will $\frac{\partial \hat{f}(x_i)}{\partial y_i}$ be high? When a small change in the observation causes a large change in the estimation($\hat{f}$)
- Can you link this to model complexity?
- Yes, indeed a complex model will be more sensitive to changes in observations whereas a simple model will be less sensitive to changes in observations
- Hence, we can say that
true error = empirical train error + small constant + $\Omega(\text{model complexity})$

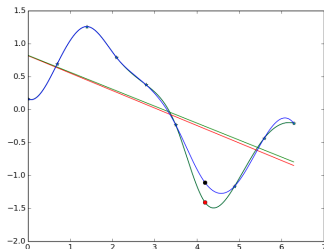
- Let us verify that indeed a complex model is more sensitive to minor changes in the data



- Let us verify that indeed a complex model is more sensitive to minor changes in the data
- We have fitted a simple and complex model for some given data



- Let us verify that indeed a complex model is more sensitive to minor changes in the data
- We have fitted a simple and complex model for some given data
- We now change one of these data points



- Let us verify that indeed a complex model is more sensitive to minor changes in the data
- We have fitted a simple and complex model for some given data
- We now change one of these data points
- The simple model does not change much as compared to the complex model

- Hence while training, instead of minimizing the training error $\mathcal{L}_{train}(\theta)$ we should minimize

$$\min_{w.r.t \theta} \mathcal{L}_{train}(\theta) + \Omega(\theta) = \mathcal{L}(\theta)$$

- Hence while training, instead of minimizing the training error $\mathcal{L}_{train}(\theta)$ we should minimize

$$\min_{w.r.t \theta} \mathcal{L}_{train}(\theta) + \Omega(\theta) = \mathcal{L}(\theta)$$

- Where $\Omega(\theta)$ would be high for complex models and small for simple models

- Hence while training, instead of minimizing the training error $\mathcal{L}_{train}(\theta)$ we should minimize

$$\min_{w.r.t \theta} \mathcal{L}_{train}(\theta) + \Omega(\theta) = \mathcal{L}(\theta)$$

- Where $\Omega(\theta)$ would be high for complex models and small for simple models
- $\Omega(\theta)$ acts as an approximate for $\frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$

- Hence while training, instead of minimizing the training error $\mathcal{L}_{train}(\theta)$ we should minimize

$$\min_{w.r.t \theta} \mathcal{L}_{train}(\theta) + \Omega(\theta) = \mathcal{L}(\theta)$$

- Where $\Omega(\theta)$ would be high for complex models and small for simple models
- $\Omega(\theta)$ acts as an approximate for $\frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$
- This is the basis for all regularization methods

- Hence while training, instead of minimizing the training error $\mathcal{L}_{train}(\theta)$ we should minimize

$$\min_{w.r.t \theta} \mathcal{L}_{train}(\theta) + \Omega(\theta) = \mathcal{L}(\theta)$$

- Where $\Omega(\theta)$ would be high for complex models and small for simple models
- $\Omega(\theta)$ acts as an approximate for $\frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$
- This is the basis for all regularization methods
- We can show that l_1 regularization, l_2 regularization, early stopping and injecting noise in input are all instances of this form of regularization.

- Why do we care about this bias variance tradeoff and model complexity?

- Why do we care about this bias variance tradeoff and model complexity?
- Deep Neural networks are highly complex models.

- Why do we care about this bias variance tradeoff and model complexity?
- Deep Neural networks are highly complex models.
- Many parameters, many non-linearities.

- Why do we care about this bias variance tradeoff and model complexity?
- Deep Neural networks are highly complex models.
- Many parameters, many non-linearities.
- It is easy for them to overfit and drive training error to 0.

- Why do we care about this bias variance tradeoff and model complexity?
- Deep Neural networks are highly complex models.
- Many parameters, many non-linearities.
- It is easy for them to overfit and drive training error to 0.
- Hence we need some form of regularization.

Different forms of regularization

- l_2 regularization

Different forms of regularization

- l_2 regularization
- Dataset augmentation

Different forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying

Different forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs

Different forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs

Different forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping

Different forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods

Different forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

Different forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

- For l_2 regularization we have,

$$\widetilde{\mathcal{L}}(w) = \mathcal{L}(w) + \frac{\alpha}{2} \|w\|^2$$

- For l_2 regularization we have,

$$\widetilde{\mathcal{L}}(w) = \mathcal{L}(w) + \frac{\alpha}{2} \|w\|^2$$

- For SGD (or its variants), we are interested in

$$\nabla \widetilde{\mathcal{L}}(w) = \nabla \mathcal{L}(w) + \alpha w$$

- For l_2 regularization we have,

$$\widetilde{\mathcal{L}}(w) = \mathcal{L}(w) + \frac{\alpha}{2} \|w\|^2$$

- For SGD (or its variants), we are interested in

$$\nabla \widetilde{\mathcal{L}}(w) = \nabla \mathcal{L}(w) + \alpha w$$

- Update rule:

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t) - \eta \alpha w_t$$

- For l_2 regularization we have,

$$\widetilde{\mathcal{L}}(w) = \mathcal{L}(w) + \frac{\alpha}{2} \|w\|^2$$

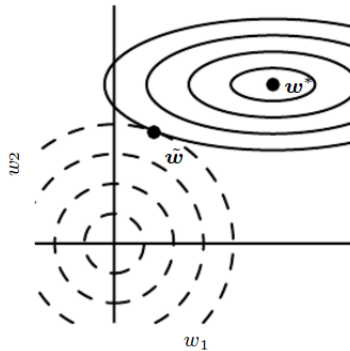
- For SGD (or its variants), we are interested in

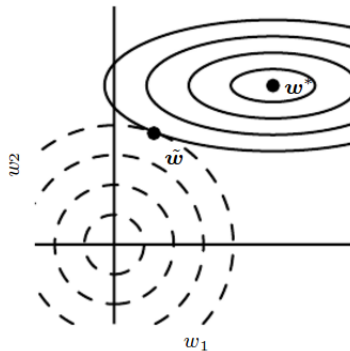
$$\nabla \widetilde{\mathcal{L}}(w) = \nabla \mathcal{L}(w) + \alpha w$$

- Update rule:

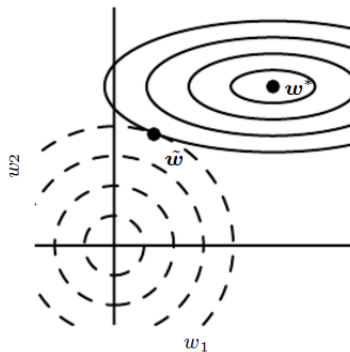
$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t) - \eta \alpha w_t$$

- Requires a very small modification to the code

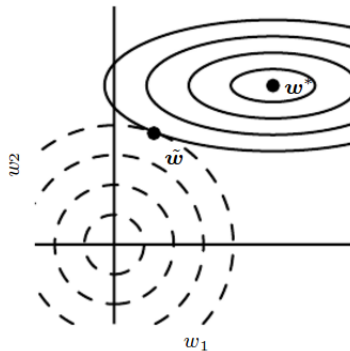




- The weight vector(w^*) is getting rotated to ($\tilde{w}$)



- The weight vector(w^*) is getting rotated to ($\tilde{w}$)
- All of its elements are shrinking but some are shrinking more than the others



- The weight vector(w^*) is getting rotated to ($\tilde{w}$)
- All of its elements are shrinking but some are shrinking more than the others
- This ensures that only important features are given high weights

Different forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout



label = 2



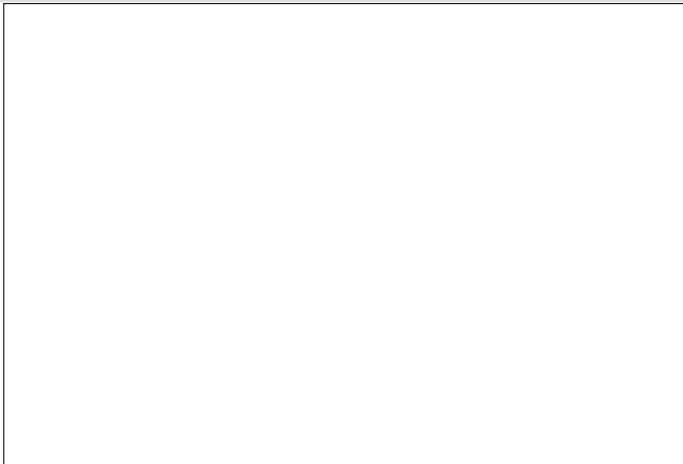
label = 2

[given training data]



label = 2

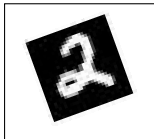
[given training data]





label = 2

[given training data]

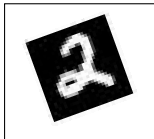


rotated by 20°



label = 2

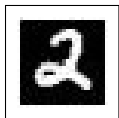
[given training data]



rotated by 20°

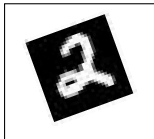


rotated by 65°



label = 2

[given training data]



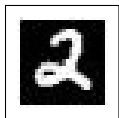
rotated by 20°



rotated by 65°

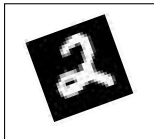


shifted vertically



label = 2

[given training data]



rotated by 20°



rotated by 65°



shifted vertically



shifted horizontally



label = 2

[given training data]



rotated by 20°



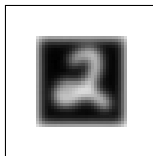
rotated by 65°



shifted vertically



shifted horizontally



blurred



label = 2

[given training data]



rotated by 20°



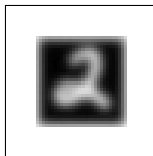
rotated by 65°



shifted vertically



shifted horizontally



blurred



changed some pixels



label = 2

[given training data]



rotated by 20°



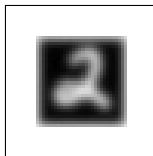
rotated by 65°



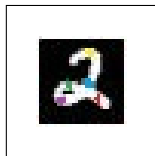
shifted vertically



shifted horizontally



blurred



changed some pixels

label = 2



label = 2

[given training data]



rotated by 20°



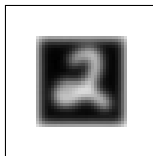
rotated by 65°



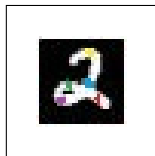
shifted vertically



shifted horizontally



blurred



changed some pixels

label = 2

[augmented data = created using some knowledge of the task]



label = 2

[given training data]
We exploit the fact that certain transformations to the image do not change the label of the image.



rotated by 20°



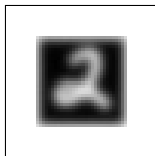
rotated by 65°



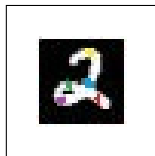
shifted vertically



shifted horizontally



blurred



changed some pixels

label = 2

[augmented data = created using some knowledge of the task]

- Typically, More data = better learning

- Typically, More data = better learning
- Works well for image classification / object recognition tasks

- Typically, More data = better learning
- Works well for image classification / object recognition tasks
- Also shown to work well for speech

- Typically, More data = better learning
- Works well for image classification / object recognition tasks
- Also shown to work well for speech
- For some tasks it may not be clear how to generate such data

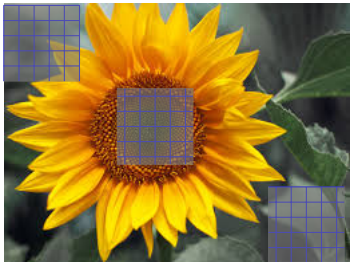
Module 8.6 : Parameter Sharing and tying

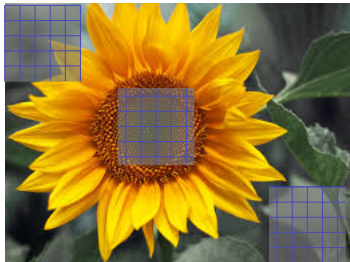
Other forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

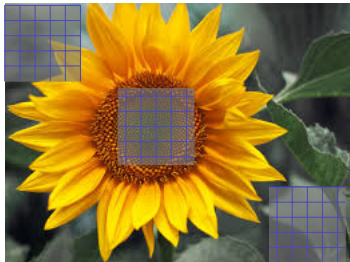
Other forms of regularization

- l_2 regularization
- Dataset augmentation
- **Parameter Sharing and tying**
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout



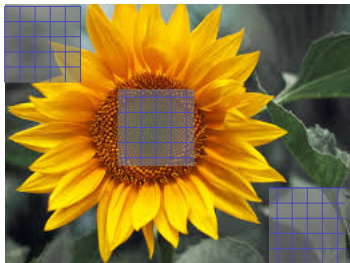


Parameter Sharing



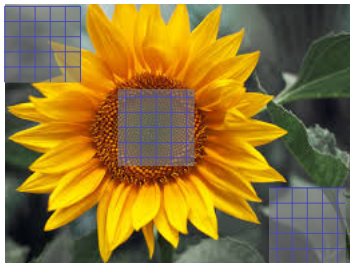
Parameter Sharing

- Used in CNNs



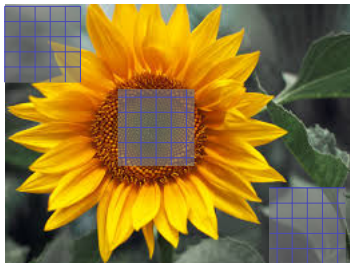
Parameter Sharing

- Used in CNNs
- Same filter applied at different positions of the image



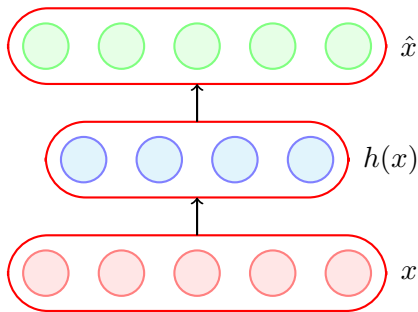
Parameter Sharing

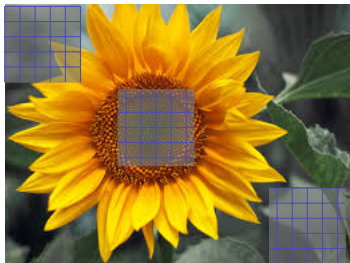
- Used in CNNs
- Same filter applied at different positions of the image
- Or same weight matrix acts on different input neurons



Parameter Sharing

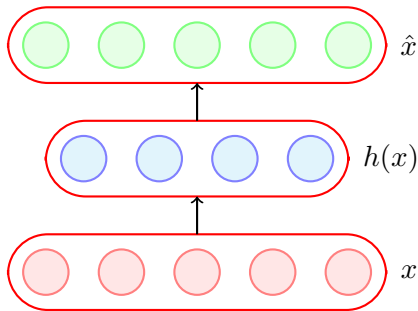
- Used in CNNs
- Same filter applied at different positions of the image
- Or same weight matrix acts on different input neurons



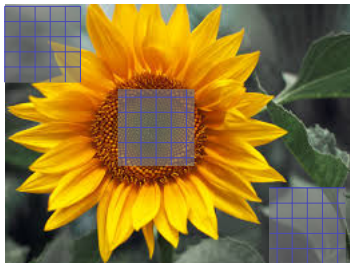


Parameter Sharing

- Used in CNNs
- Same filter applied at different positions of the image
- Or same weight matrix acts on different input neurons

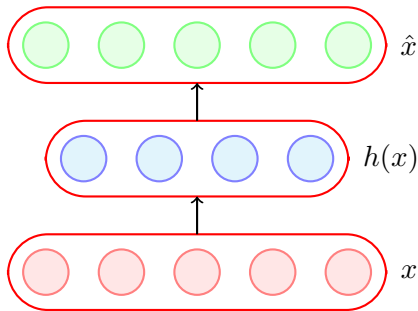


Parameter Tying



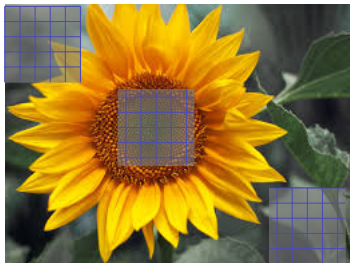
Parameter Sharing

- Used in CNNs
- Same filter applied at different positions of the image
- Or same weight matrix acts on different input neurons



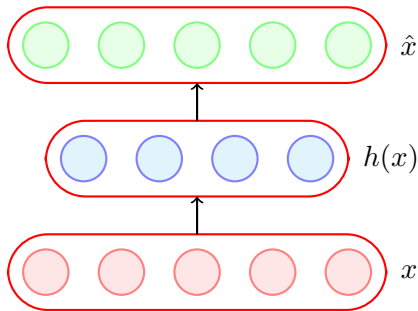
Parameter Tying

- Typically used in autoencoders



Parameter Sharing

- Used in CNNs
- Same filter applied at different positions of the image
- Or same weight matrix acts on different input neurons



Parameter Tying

- Typically used in autoencoders
- The encoder and decoder weights are tied.

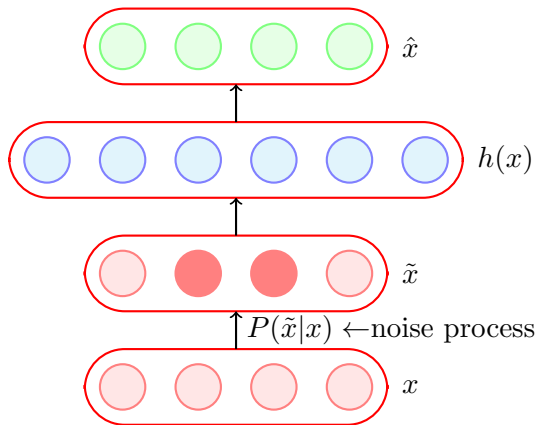
Module 8.7 : Adding Noise to the inputs

Other forms of regularization

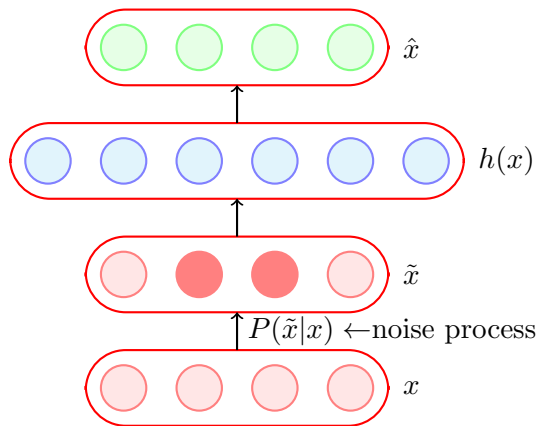
- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

Other forms of regularization

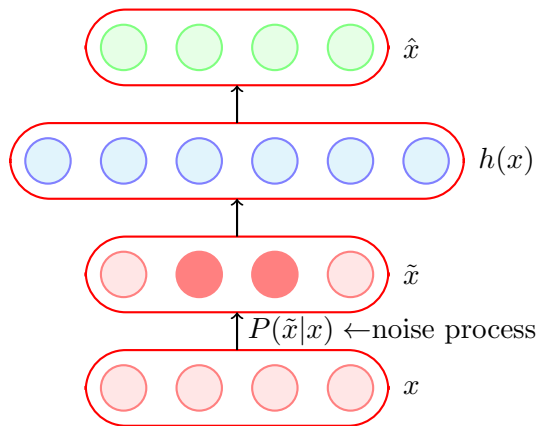
- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

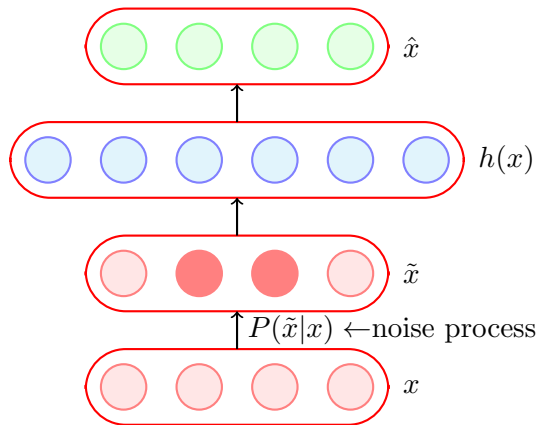


- We saw this in Autoencoder

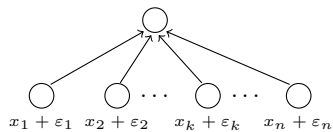


- We saw this in Autoencoder
- We can show that for a simple input output neural network, adding Gaussian noise to the input is equivalent to weight decay (L_2 regularisation)

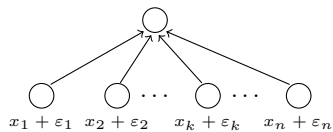




- We saw this in Autoencoder
- We can show that for a simple input output neural network, adding Gaussian noise to the input is equivalent to weight decay (L_2 regularisation)
- Can be viewed as data augmentation

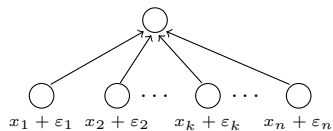


$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$



$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

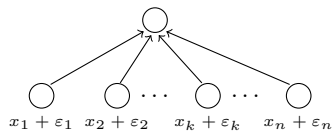
$$\tilde{x}_i = x_i + \epsilon_i$$



$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\tilde{x}_i = x_i + \epsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

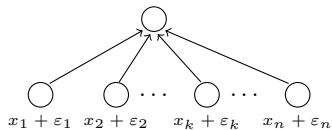


$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\tilde{x}_i = x_i + \varepsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$



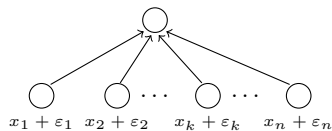
$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\tilde{x}_i = x_i + \epsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$

$$= \sum_{i=1}^n w_i x_i + \sum_{i=1}^n w_i \epsilon_i$$



$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

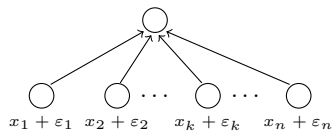
$$\tilde{x}_i = x_i + \epsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$

$$= \sum_{i=1}^n w_i x_i + \sum_{i=1}^n w_i \epsilon_i$$

$$= \hat{y} + \sum_{i=1}^n w_i \epsilon_i$$



We are interested in $E[(\tilde{y} - y)^2]$

$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

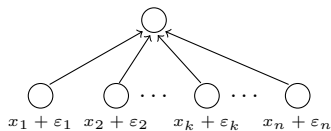
$$\tilde{x}_i = x_i + \epsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$

$$= \sum_{i=1}^n w_i x_i + \sum_{i=1}^n w_i \epsilon_i$$

$$= \hat{y} + \sum_{i=1}^n w_i \epsilon_i$$



$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\tilde{x}_i = x_i + \varepsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

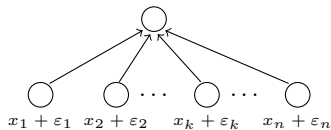
$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$

$$= \sum_{i=1}^n w_i x_i + \sum_{i=1}^n w_i \varepsilon_i$$

$$= \hat{y} + \sum_{i=1}^n w_i \varepsilon_i$$

We are interested in $E[(\tilde{y} - y)^2]$

$$E[(\tilde{y} - y)^2] = E\left[\left(\hat{y} + \sum_{i=1}^n w_i \varepsilon_i - y\right)^2\right]$$



$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\tilde{x}_i = x_i + \varepsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

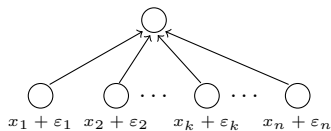
$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$

$$= \sum_{i=1}^n w_i x_i + \sum_{i=1}^n w_i \varepsilon_i$$

$$= \hat{y} + \sum_{i=1}^n w_i \varepsilon_i$$

We are interested in $E[(\tilde{y} - y)^2]$

$$\begin{aligned} E[(\tilde{y} - y)^2] &= E\left[\left(\hat{y} + \sum_{i=1}^n w_i \varepsilon_i - y\right)^2\right] \\ &= E\left[\left((\hat{y} - y) + \left(\sum_{i=1}^n w_i \varepsilon_i\right)\right)^2\right] \end{aligned}$$



$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\tilde{x}_i = x_i + \varepsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

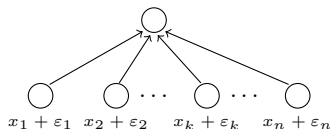
$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$

$$= \sum_{i=1}^n w_i x_i + \sum_{i=1}^n w_i \varepsilon_i$$

$$= \hat{y} + \sum_{i=1}^n w_i \varepsilon_i$$

We are interested in $E[(\tilde{y} - y)^2]$

$$\begin{aligned} E[(\tilde{y} - y)^2] &= E\left[\left(\hat{y} + \sum_{i=1}^n w_i \varepsilon_i - y\right)^2\right] \\ &= E\left[\left((\hat{y} - y) + \left(\sum_{i=1}^n w_i \varepsilon_i\right)\right)^2\right] \\ &= E[(\hat{y} - y)^2] + E\left[2(\hat{y} - y) \sum_{i=1}^n w_i \varepsilon_i\right] + E\left[\left(\sum_{i=1}^n w_i \varepsilon_i\right)^2\right] \end{aligned}$$



$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\tilde{x}_i = x_i + \varepsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

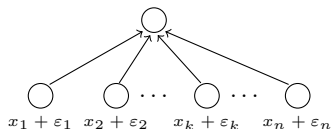
$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$

$$= \sum_{i=1}^n w_i x_i + \sum_{i=1}^n w_i \varepsilon_i$$

$$= \hat{y} + \sum_{i=1}^n w_i \varepsilon_i$$

We are interested in $E[(\tilde{y} - y)^2]$

$$\begin{aligned} E[(\tilde{y} - y)^2] &= E\left[\left(\hat{y} + \sum_{i=1}^n w_i \varepsilon_i - y\right)^2\right] \\ &= E\left[\left((\hat{y} - y) + \left(\sum_{i=1}^n w_i \varepsilon_i\right)\right)^2\right] \\ &= E[(\hat{y} - y)^2] + E\left[2(\hat{y} - y) \sum_{i=1}^n w_i \varepsilon_i\right] + E\left[\left(\sum_{i=1}^n w_i \varepsilon_i\right)^2\right] \\ &= E[(\hat{y} - y)^2] + 0 + E\left[\sum_{i=1}^n w_i^2 \varepsilon_i^2\right] \\ &(\because \varepsilon_i \text{ is independent of } \varepsilon_j \text{ and } \varepsilon_i \text{ is independent of } (\hat{y} - y)) \end{aligned}$$



$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\tilde{x}_i = x_i + \epsilon_i$$

$$\hat{y} = \sum_{i=1}^n w_i x_i$$

$$\tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i$$

$$= \sum_{i=1}^n w_i x_i + \sum_{i=1}^n w_i \epsilon_i$$

$$= \hat{y} + \sum_{i=1}^n w_i \epsilon_i$$

We are interested in $E[(\tilde{y} - y)^2]$

$$E[(\tilde{y} - y)^2] = E\left[\left(\hat{y} + \sum_{i=1}^n w_i \epsilon_i - y\right)^2\right]$$

$$= E\left[\left((\hat{y} - y) + \left(\sum_{i=1}^n w_i \epsilon_i\right)\right)^2\right]$$

$$= E[(\hat{y} - y)^2] + E\left[2(\hat{y} - y) \sum_{i=1}^n w_i \epsilon_i\right] + E\left[\left(\sum_{i=1}^n w_i \epsilon_i\right)^2\right]$$

$$= E[(\hat{y} - y)^2] + 0 + E\left[\sum_{i=1}^n w_i^2 \epsilon_i^2\right]$$

($\because \epsilon_i$ is independent of ϵ_j and ϵ_i is independent of $(\hat{y} - y)$)

$$= (E[(\hat{y} - y)^2] + \sigma^2 \sum_{i=1}^n w_i^2) \quad (\text{same as } L_2 \text{ norm penalty})$$

Module 8.8 : Adding Noise to the outputs

Other forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout



0	0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

Hard targets



0	0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

Hard targets

$$\text{minimize : } \sum_{i=0}^9 p_i \log q_i$$



0	0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

Hard targets

$$\text{minimize : } \sum_{i=0}^9 p_i \log q_i$$

true distribution : $p = \{0, 0, 1, 0, 0, 0, 0, 0, 0, 0\}$



0	0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

Hard targets

$$\text{minimize : } \sum_{i=0}^9 p_i \log q_i$$

true distribution : $p = \{0, 0, 1, 0, 0, 0, 0, 0, 0, 0\}$

estimated distribution : q



0	0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

Hard targets

$$\text{minimize : } \sum_{i=0}^9 p_i \log q_i$$

true distribution : $p = \{0, 0, 1, 0, 0, 0, 0, 0, 0, 0\}$

estimated distribution : q

Intuition

- Do not trust the true labels, they may be noisy



0	0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

Hard targets

$$\text{minimize : } \sum_{i=0}^9 p_i \log q_i$$

true distribution : $p = \{0, 0, 1, 0, 0, 0, 0, 0, 0, 0\}$

estimated distribution : q

Intuition

- Do not trust the true labels, they may be noisy
- Instead, use soft targets

[illegible]

Soft targets

[illegible]

Soft targets

 $\varepsilon = \text{small positive constant}$



$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$1 - \epsilon$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$
----------------------	----------------------	----------------	----------------------	----------------------	----------------------	----------------------	----------------------	----------------------	----------------------

Soft targets

$\epsilon = \text{small positive constant}$

$$\text{minimize : } \sum_{i=0}^9 p_i \log q_i$$



$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$1 - \epsilon$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$	$\frac{\epsilon}{9}$
----------------------	----------------------	----------------	----------------------	----------------------	----------------------	----------------------	----------------------	----------------------	----------------------

Soft targets

ϵ = small positive constant

$$\text{minimize : } \sum_{i=0}^9 p_i \log q_i$$

$$\text{true distribution + noise : } p = \left\{ \frac{\epsilon}{9}, \frac{\epsilon}{9}, 1 - \epsilon, \frac{\epsilon}{9}, \dots \right\}$$



$\frac{\varepsilon}{9}$	$\frac{\varepsilon}{9}$	$1 - \varepsilon$	$\frac{\varepsilon}{9}$	$\frac{\varepsilon}{9}$	$\frac{\varepsilon}{9}$	$\frac{\varepsilon}{9}$	$\frac{\varepsilon}{9}$	$\frac{\varepsilon}{9}$	$\frac{\varepsilon}{9}$
-------------------------	-------------------------	-------------------	-------------------------	-------------------------	-------------------------	-------------------------	-------------------------	-------------------------	-------------------------

Soft targets

$\varepsilon = \text{small positive constant}$

$$\text{minimize : } \sum_{i=0}^9 p_i \log q_i$$

$$\text{true distribution + noise : } p = \left\{ \frac{\varepsilon}{9}, \frac{\varepsilon}{9}, 1 - \varepsilon, \frac{\varepsilon}{9}, \dots \right\}$$

$$\text{estimated distribution : } q$$

Module 8.9 : Early stopping

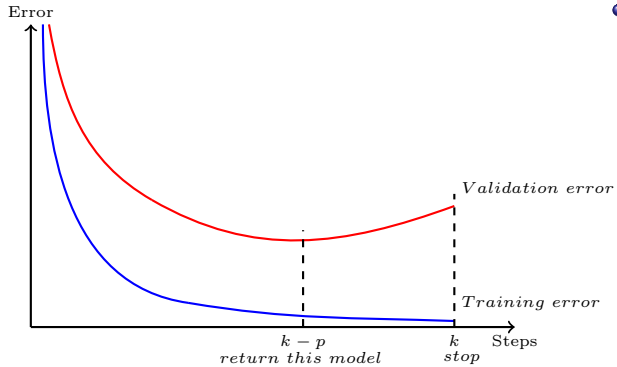
Other forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

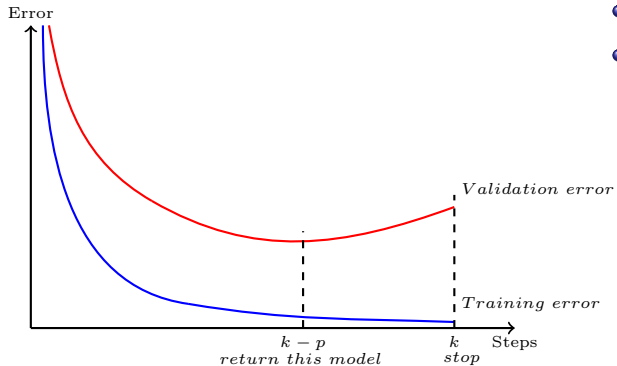
Other forms of regularization

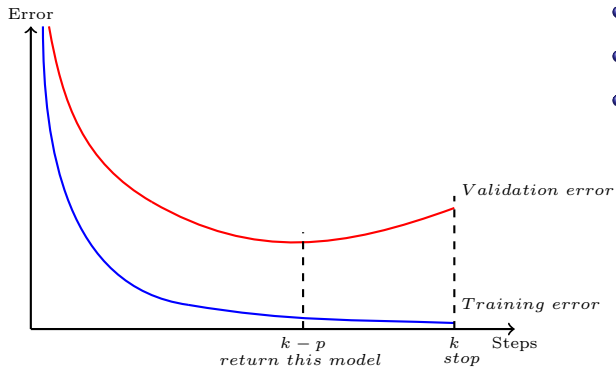
- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

- Track the validation error

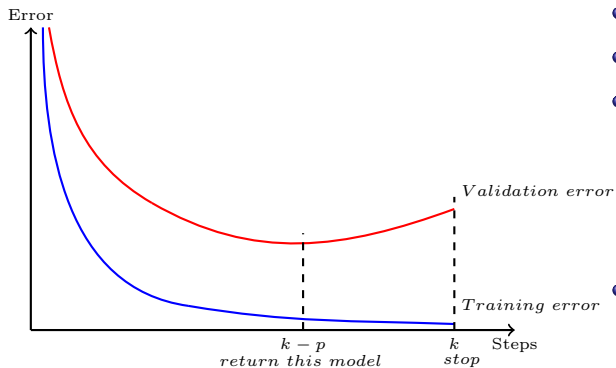


- Track the validation error
- Have a patience parameter p



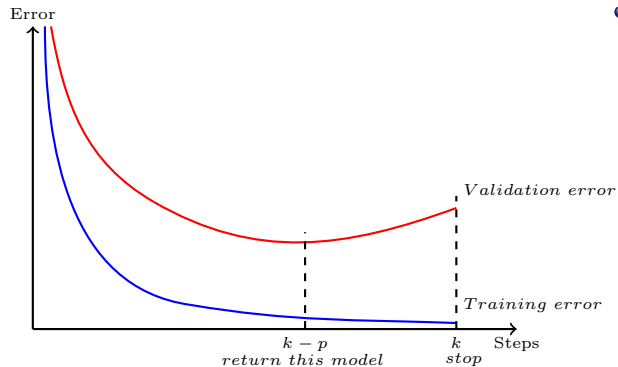


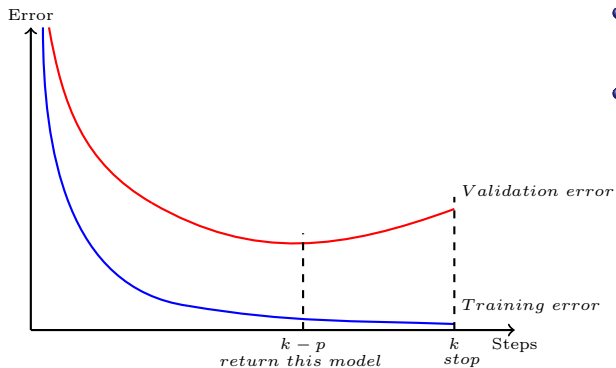
- Track the validation error
- Have a patience parameter p
- If you are at step k and there was no improvement in validation error in the previous p steps then stop training and return the model stored at step $k - p$



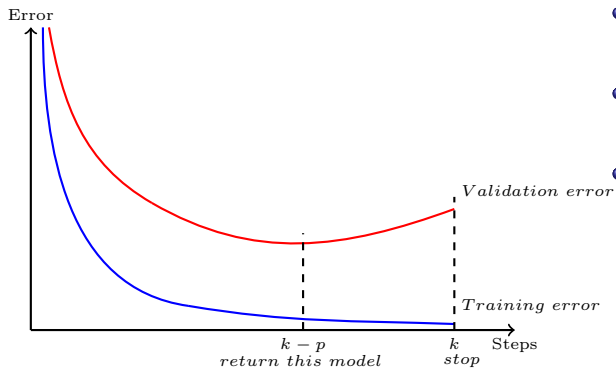
- Track the validation error
- Have a patience parameter p
- If you are at step k and there was no improvement in validation error in the previous p steps then stop training and return the model stored at step $k - p$
- Basically, stop the training early before it drives the training error to 0 and blows up the validation error

- Very effective and the mostly widely used form of regularization

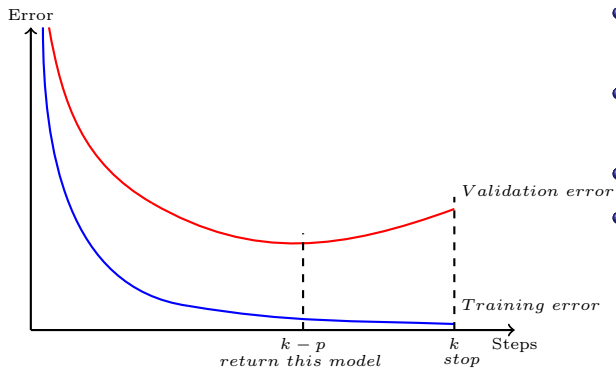




- Very effective and the mostly widely used form of regularization
- Can be used even with other regularizers (such as l_2)

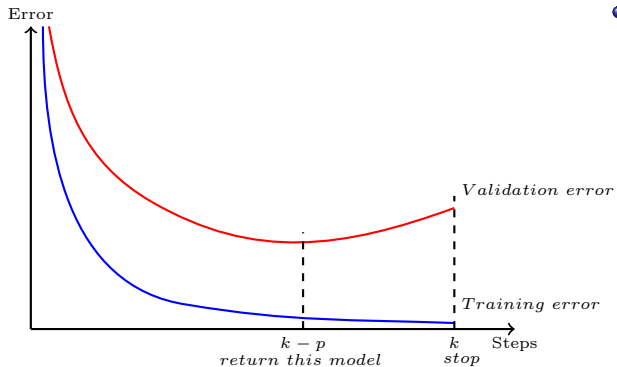


- Very effective and the mostly widely used form of regularization
- Can be used even with other regularizers (such as l_2)
- How does it act as a regularizer ?



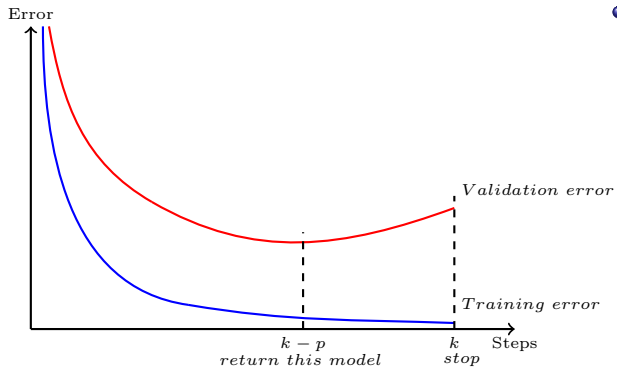
- Very effective and the mostly widely used form of regularization
- Can be used even with other regularizers (such as l_2)
- How does it act as a regularizer ?
- We will first see an intuitive explanation and then a mathematical analysis

- Recall that the update rule in SGD is



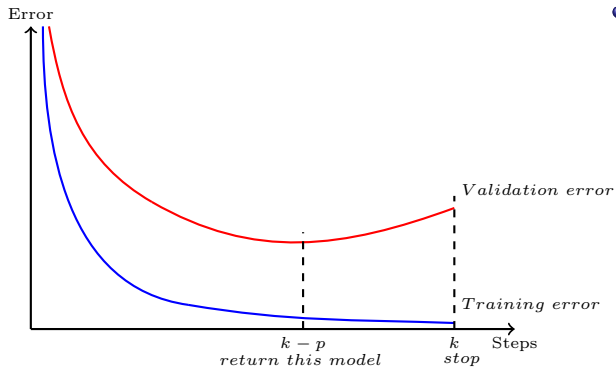
- Recall that the update rule in SGD is

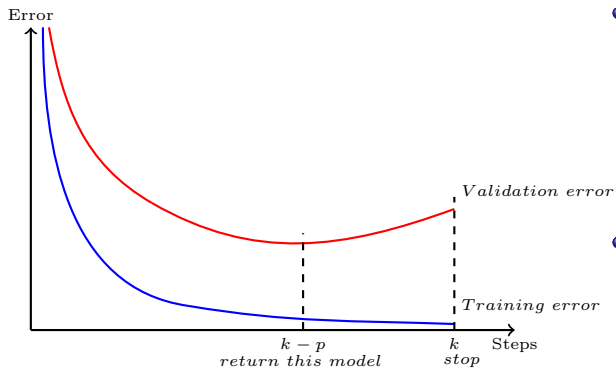
$$w_{t+1} = w_t - \eta \nabla w_t$$



- Recall that the update rule in SGD is

$$\begin{aligned}w_{t+1} &= w_t - \eta \nabla w_t \\ &= w_0 - \eta \sum_{i=1}^t \nabla w_i\end{aligned}$$

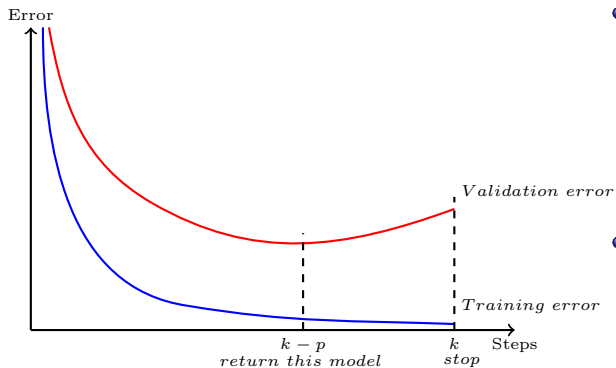




- Recall that the update rule in SGD is

$$\begin{aligned}w_{t+1} &= w_t - \eta \nabla w_t \\ &= w_0 - \eta \sum_{i=1}^t \nabla w_i\end{aligned}$$

- Let τ be the maximum value of ∇w_i then

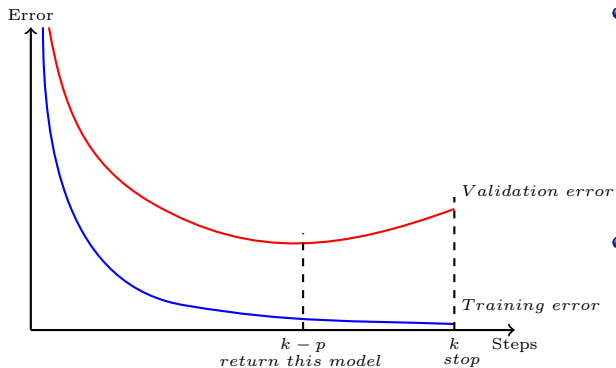


- Recall that the update rule in SGD is

$$\begin{aligned}
 w_{t+1} &= w_t - \eta \nabla w_t \\
 &= w_0 - \eta \sum_{i=1}^t \nabla w_i
 \end{aligned}$$

- Let τ be the maximum value of ∇w_i then

$$|w_{t+1} - w_0| \leq \eta t |\tau|$$



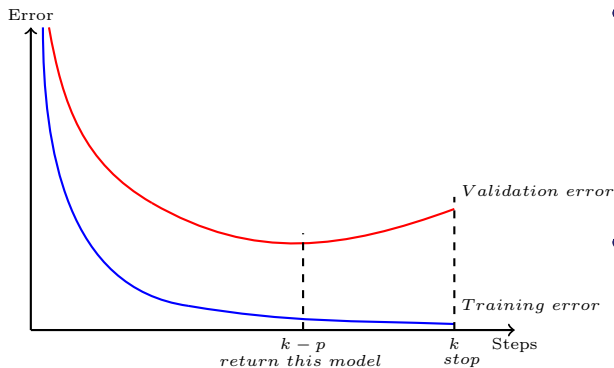
- Recall that the update rule in SGD is

$$\begin{aligned} w_{t+1} &= w_t - \eta \nabla w_t \\ &= w_0 - \eta \sum_{i=1}^t \nabla w_i \end{aligned}$$

- Let τ be the maximum value of ∇w_i then

$$|w_{t+1} - w_0| \leq \eta t |\tau|$$

- Thus, t controls how far w_t can go from the initial w_0



- Recall that the update rule in SGD is

$$\begin{aligned} w_{t+1} &= w_t - \eta \nabla w_t \\ &= w_0 - \eta \sum_{i=1}^t \nabla w_i \end{aligned}$$

- Let τ be the maximum value of ∇w_i then

$$|w_{t+1} - w_0| \leq \eta t |\tau|$$

- Thus, t controls how far w_t can go from the initial w_0
- In other words it controls the space of exploration

Things to be remember

- Early stopping only allows t updates to the parameters.

Things to be remember

- Early stopping only allows t updates to the parameters.
- If a parameter w corresponds to a dimension which is important for the loss $\mathcal{L}(\theta)$ then $\frac{\partial \mathcal{L}(\theta)}{\partial w}$ will be large

Things to be remember

- Early stopping only allows t updates to the parameters.
- If a parameter w corresponds to a dimension which is important for the loss $\mathcal{L}(\theta)$ then $\frac{\partial \mathcal{L}(\theta)}{\partial w}$ will be large

Things to be remember

- Early stopping only allows t updates to the parameters.
- If a parameter w corresponds to a dimension which is important for the loss $\mathcal{L}(\theta)$ then $\frac{\partial \mathcal{L}(\theta)}{\partial w}$ will be large
- However if a parameter is not important ($\frac{\partial \mathcal{L}(\theta)}{\partial w}$ is small) then its updates will be small and the parameter will not be able to grow large in ' t ' steps

Things to be remember

- Early stopping only allows t updates to the parameters.
- If a parameter w corresponds to a dimension which is important for the loss $\mathcal{L}(\theta)$ then $\frac{\partial \mathcal{L}(\theta)}{\partial w}$ will be large
- However if a parameter is not important ($\frac{\partial \mathcal{L}(\theta)}{\partial w}$ is small) then its updates will be small and the parameter will not be able to grow large in ' t ' steps
- Early stopping will thus effectively shrink the parameters corresponding to less important directions (same as weight decay).

Module 8.10 : Ensemble methods

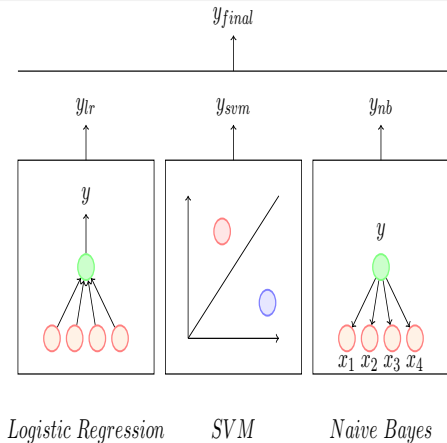
Other forms of regularization

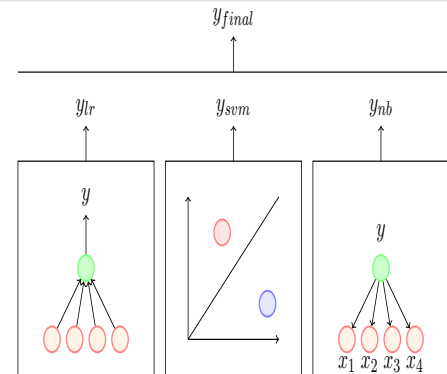
- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

Other forms of regularization

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

- Combine the output of different models to reduce generalization error



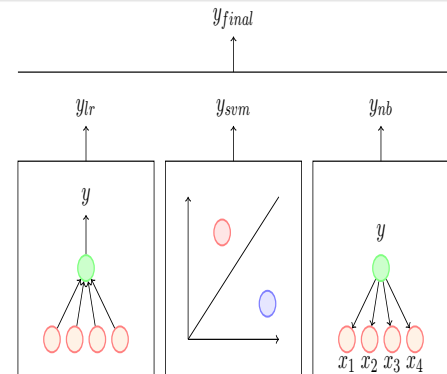


Logistic Regression

SVM

Naive Bayes

- Combine the output of different models to reduce generalization error
- The models can correspond to different classifiers

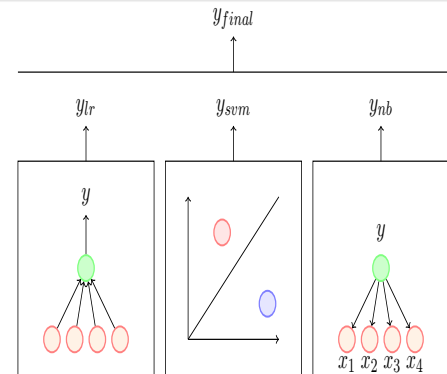


Logistic Regression

SVM

Naive Bayes

- Combine the output of different models to reduce generalization error
- The models can correspond to different classifiers
- It could be different instances of the same classifier trained with:

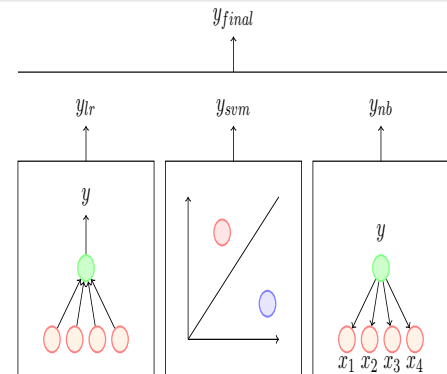


Logistic Regression

SVM

Naive Bayes

- Combine the output of different models to reduce generalization error
- The models can correspond to different classifiers
- It could be different instances of the same classifier trained with:
 - different hyperparameters

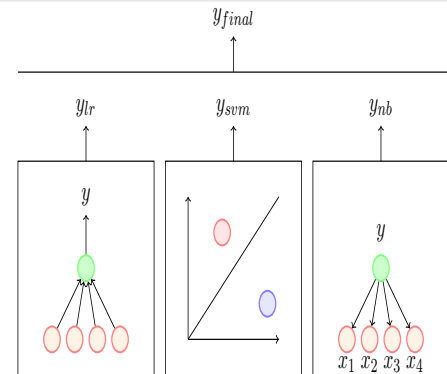


Logistic Regression

SVM

Naive Bayes

- Combine the output of different models to reduce generalization error
- The models can correspond to different classifiers
- It could be different instances of the same classifier trained with:
 - different hyperparameters
 - different features

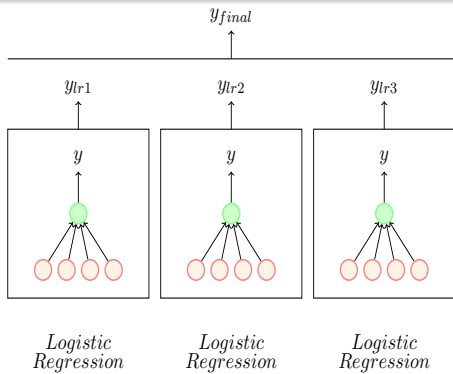


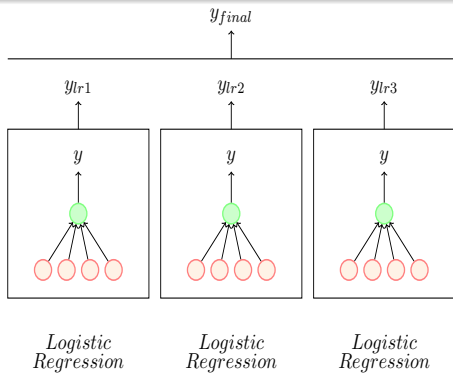
Logistic Regression

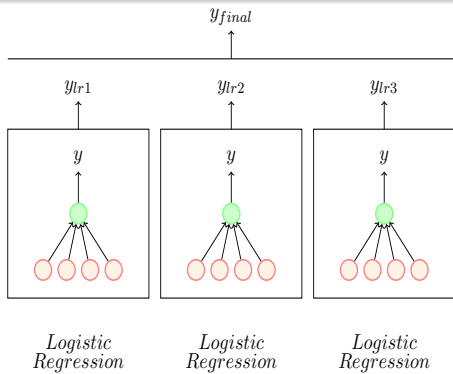
SVM

Naive Bayes

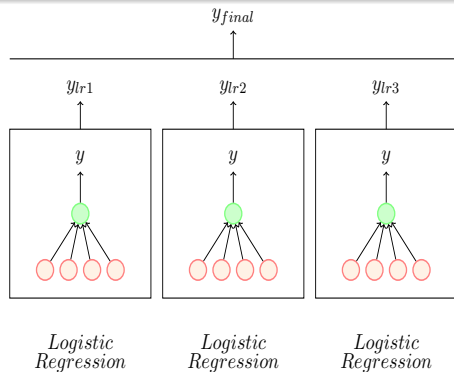
- Combine the output of different models to reduce generalization error
- The models can correspond to different classifiers
- It could be different instances of the same classifier trained with:
 - different hyperparameters
 - different features
 - different samples of the training data



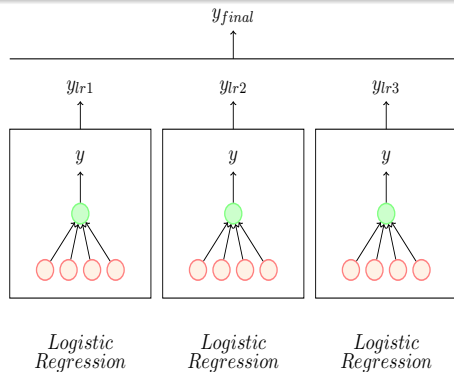




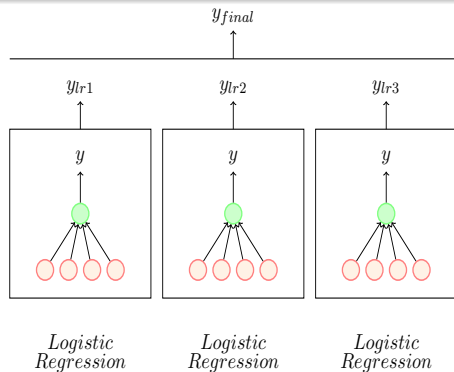
- Bagging: form an ensemble using different instances of the same classifier



- Bagging: form an ensemble using different instances of the same classifier
- From a given dataset, construct multiple training sets by sampling with replacement ($T_1, T_2, \dots, T_k$)



- Bagging: form an ensemble using different instances of the same classifier
- From a given dataset, construct multiple training sets by sampling with replacement ($T_1, T_2, \dots, T_k$)
- Train i^{th} instance of the classifier using training set T_i



Each model trained with a different sample of the data (sampling with replacement)

- Bagging: form an ensemble using different instances of the same classifier
- From a given dataset, construct multiple training sets by sampling with replacement ($T_1, T_2, \dots, T_k$)
- Train i^{th} instance of the classifier using training set T_i

- When would bagging work?

- When would bagging work?
- Consider a set of k LR models

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- $Variance = E[\varepsilon_i^2] = V$

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- $Variance = E[\varepsilon_i^2] = V$
- $Covariance = E[\varepsilon_i \varepsilon_j] = C$

- The error made by the average prediction of all the models is $\frac{1}{k} \sum_i \varepsilon_i$

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- $Variance = E[\varepsilon_i^2] = V$
- $Covariance = E[\varepsilon_i \varepsilon_j] = C$

- The error made by the average prediction of all the models is $\frac{1}{k} \sum_i \varepsilon_i$
- The expected squared error is :

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- $Variance = E[\varepsilon_i^2] = V$
- $Covariance = E[\varepsilon_i \varepsilon_j] = C$

- The error made by the average prediction of all the models is $\frac{1}{k} \sum_i \varepsilon_i$
- The expected squared error is :

$$mse = E[(\frac{1}{k} \sum_i \varepsilon_i)^2]$$

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- *Variance* = $E[\varepsilon_i^2] = V$
- *Covariance* = $E[\varepsilon_i \varepsilon_j] = C$

- The error made by the average prediction of all the models is $\frac{1}{k} \sum_i \varepsilon_i$
- The expected squared error is :

$$\begin{aligned} mse &= E[(\frac{1}{k} \sum_i \varepsilon_i)^2] \\ &= \frac{1}{k^2} E[\sum_i \sum_{i=j} \varepsilon_i \varepsilon_j + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j] \end{aligned}$$

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- *Variance* = $E[\varepsilon_i^2] = V$
- *Covariance* = $E[\varepsilon_i \varepsilon_j] = C$

- The error made by the average prediction of all the models is $\frac{1}{k} \sum_i \varepsilon_i$
- The expected squared error is :

$$\begin{aligned}
 mse &= E\left[\left(\frac{1}{k} \sum_i \varepsilon_i\right)^2\right] \\
 &= \frac{1}{k^2} E\left[\sum_i \sum_{i=j} \varepsilon_i \varepsilon_j + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j\right] \\
 &= \frac{1}{k^2} E\left[\sum_i \varepsilon_i^2 + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j\right]
 \end{aligned}$$

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- *Variance* = $E[\varepsilon_i^2] = V$
- *Covariance* = $E[\varepsilon_i \varepsilon_j] = C$

- The error made by the average prediction of all the models is $\frac{1}{k} \sum_i \varepsilon_i$
- The expected squared error is :

$$\begin{aligned}
 mse &= E\left[\left(\frac{1}{k} \sum_i \varepsilon_i\right)^2\right] \\
 &= \frac{1}{k^2} E\left[\sum_i \sum_{i=j} \varepsilon_i \varepsilon_j + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j\right] \\
 &= \frac{1}{k^2} E\left[\sum_i \varepsilon_i^2 + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j\right] \\
 &= \frac{1}{k^2} \left(\sum_i E[\varepsilon_i^2] + \sum_i \sum_{i \neq j} E[\varepsilon_i \varepsilon_j]\right)
 \end{aligned}$$

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- $Variance = E[\varepsilon_i^2] = V$
- $Covariance = E[\varepsilon_i \varepsilon_j] = C$

- The error made by the average prediction of all the models is $\frac{1}{k} \sum_i \varepsilon_i$
- The expected squared error is :

$$\begin{aligned}
 mse &= E\left[\left(\frac{1}{k} \sum_i \varepsilon_i\right)^2\right] \\
 &= \frac{1}{k^2} E\left[\sum_i \sum_{i=j} \varepsilon_i \varepsilon_j + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j\right] \\
 &= \frac{1}{k^2} E\left[\sum_i \varepsilon_i^2 + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j\right] \\
 &= \frac{1}{k^2} \left(\sum_i E[\varepsilon_i^2] + \sum_i \sum_{i \neq j} E[\varepsilon_i \varepsilon_j]\right) \\
 &= \frac{1}{k^2} (kV + k(k-1)C)
 \end{aligned}$$

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- $Variance = E[\varepsilon_i^2] = V$
- $Covariance = E[\varepsilon_i \varepsilon_j] = C$

- The error made by the average prediction of all the models is $\frac{1}{k} \sum_i \varepsilon_i$
- The expected squared error is :

$$\begin{aligned}
 mse &= E\left[\left(\frac{1}{k} \sum_i \varepsilon_i\right)^2\right] \\
 &= \frac{1}{k^2} E\left[\sum_i \sum_{i=j} \varepsilon_i \varepsilon_j + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j\right] \\
 &= \frac{1}{k^2} E\left[\sum_i \varepsilon_i^2 + \sum_i \sum_{i \neq j} \varepsilon_i \varepsilon_j\right] \\
 &= \frac{1}{k^2} \left(\sum_i E[\varepsilon_i^2] + \sum_i \sum_{i \neq j} E[\varepsilon_i \varepsilon_j]\right) \\
 &= \frac{1}{k^2} (kV + k(k-1)C) \\
 &= \frac{1}{k} V + \frac{k-1}{k} C
 \end{aligned}$$

- When would bagging work?
- Consider a set of k LR models
- Suppose that each model makes an error ε_i on a test example
- Let ε_i be drawn from a zero mean multivariate normal distribution
- $Variance = E[\varepsilon_i^2] = V$
- $Covariance = E[\varepsilon_i \varepsilon_j] = C$

$$mse = \frac{1}{k}V + \frac{k-1}{k}C$$

$$mse = \frac{1}{k}V + \frac{k-1}{k}C$$

- When would bagging work ?

$$mse = \frac{1}{k}V + \frac{k-1}{k}C$$

- When would bagging work ?
- If the errors of the model are perfectly correlated then $V = C$ and $mse = V$ [bagging does not help: the mse of the ensemble is as bad as the individual models]

$$mse = \frac{1}{k}V + \frac{k-1}{k}C$$

- When would bagging work ?
- If the errors of the model are perfectly correlated then $V = C$ and $mse = V$ [bagging does not help: the mse of the ensemble is as bad as the individual models]
- If the errors of the model are independent or uncorrelated then $C = 0$ and the mse of the ensemble reduces to $\frac{1}{k}V$

$$mse = \frac{1}{k}V + \frac{k-1}{k}C$$

- When would bagging work ?
- If the errors of the model are perfectly correlated then $V = C$ and $mse = V$ [bagging does not help: the mse of the ensemble is as bad as the individual models]
- If the errors of the model are independent or uncorrelated then $C = 0$ and the mse of the ensemble reduces to $\frac{1}{k}V$
- On average, the ensemble will perform at least as well as its individual members

Module 8.11 : Dropout

Other forms of regularization

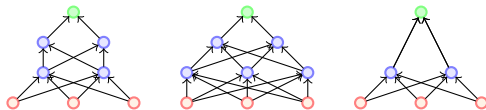
- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

Other forms of regularization

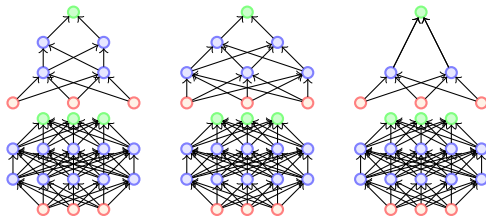
- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout

- Typically model averaging (bagging ensemble) always helps

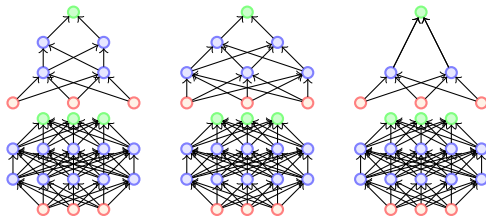
- Typically model averaging (bagging ensemble) always helps
- Training several large neural networks for making an ensemble is prohibitively expensive



- Typically model averaging (bagging ensemble) always helps
- Training several large neural networks for making an ensemble is prohibitively expensive
- Option 1: Train several neural networks having different architectures (obviously expensive)

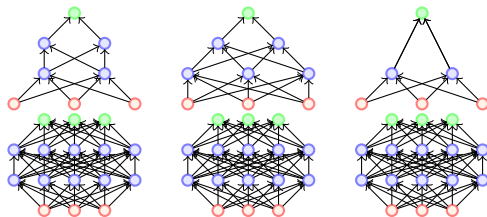


- Typically model averaging (bagging ensemble) always helps
- Training several large neural networks for making an ensemble is prohibitively expensive
- Option 1: Train several neural networks having different architectures (obviously expensive)
- Option 2: Train multiple instances of the same network using different training samples (again expensive)

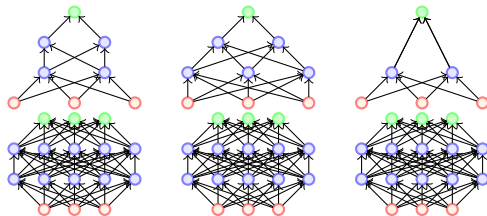


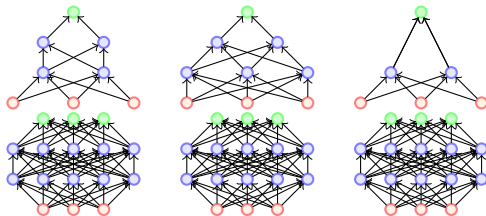
- Typically model averaging(bagging ensemble) always helps
- Training several large neural networks for making an ensemble is prohibitively expensive
- Option 1: Train several neural networks having different architectures(Obviously expensive)
- Option 2: Train multiple instances of the same network using different training samples (again expensive)
- Even if we manage to train with option 1 or option 2, combining several models at test time is infeasible in real time applications

- Dropout is a technique which addresses both these issues.

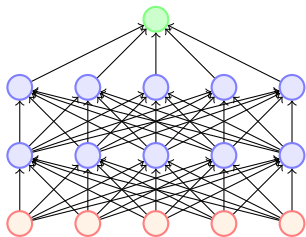


- Dropout is a technique which addresses both these issues.
- Effectively it allows training several neural networks without any significant computational overhead.

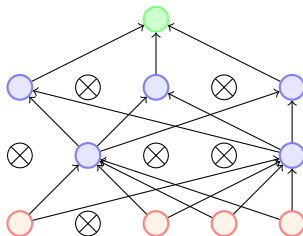
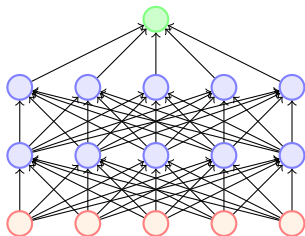




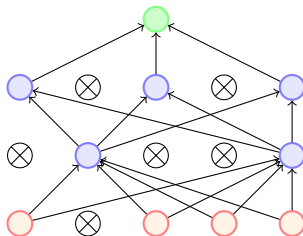
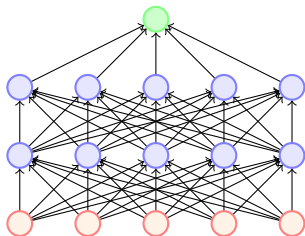
- Dropout is a technique which addresses both these issues.
- Effectively it allows training several neural networks without any significant computational overhead.
- Also gives an efficient approximate way of combining exponentially many different neural networks.



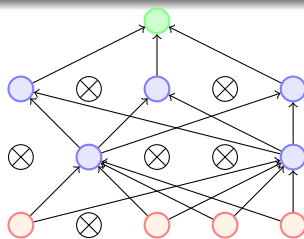
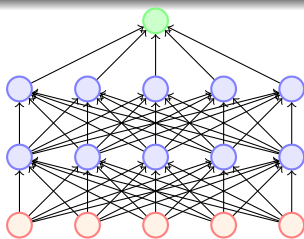
- Dropout refers to dropping out units

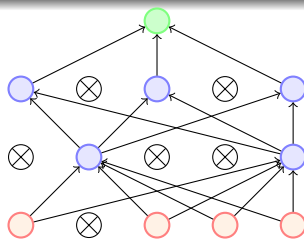
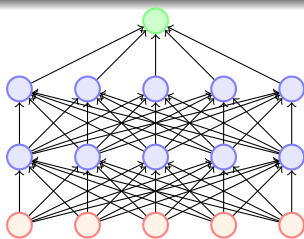


- Dropout refers to dropping out units
- Temporarily remove a node and all its incoming/outgoing connections resulting in a thinned network

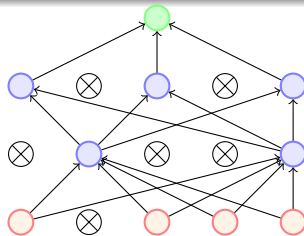
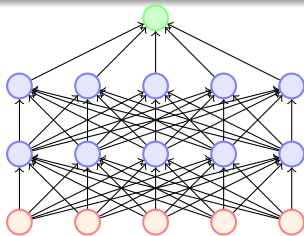


- Dropout refers to dropping out units
- Temporarily remove a node and all its incoming/outgoing connections resulting in a thinned network
- Each node is retained with a fixed probability (typically $p = 0.5$) for hidden nodes and $p = 0.8$ for visible nodes

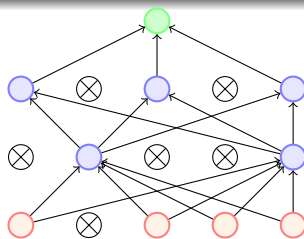
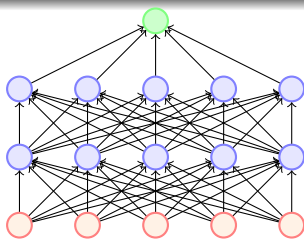




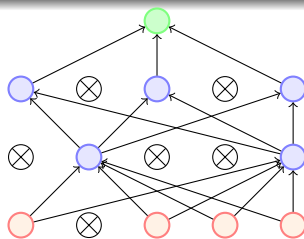
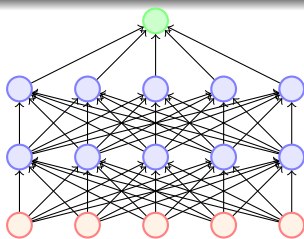
- Suppose a neural network has n nodes



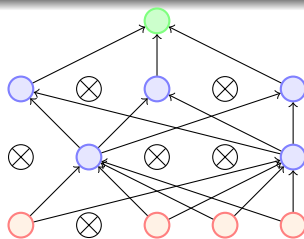
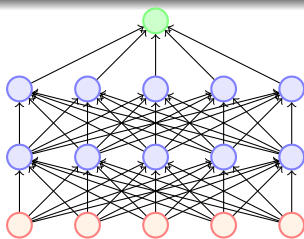
- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped



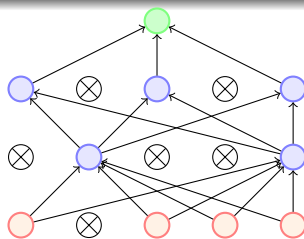
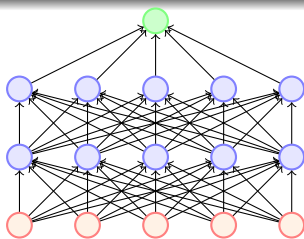
- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network



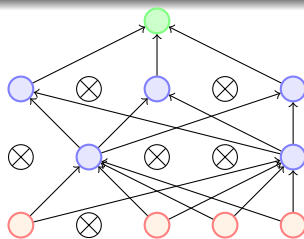
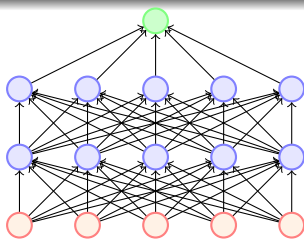
- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network
- Given a total of n nodes, what are the total number of thinned networks that can be formed?



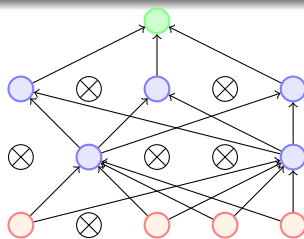
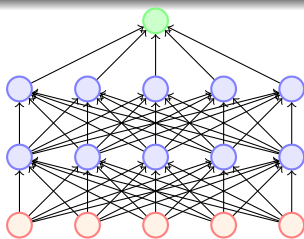
- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network
- Given a total of n nodes, what are the total number of thinned networks that can be formed? 2^n



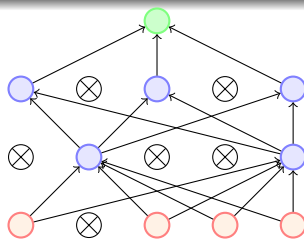
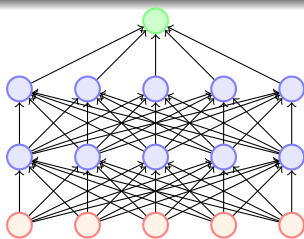
- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network
- Given a total of n nodes, what are the total number of thinned networks that can be formed? 2^n
- Of course, this is prohibitively large and we cannot possibly train so many networks



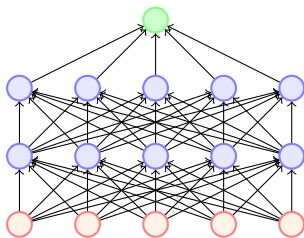
- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network
- Given a total of n nodes, what are the total number of thinned networks that can be formed? 2^n
- Of course, this is prohibitively large and we cannot possibly train so many networks
- **Trick:** (1) Share the weights across all the networks

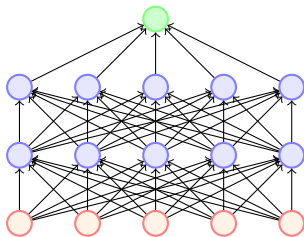


- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network
- Given a total of n nodes, what are the total number of thinned networks that can be formed? 2^n
- Of course, this is prohibitively large and we cannot possibly train so many networks
- **Trick:** (1) Share the weights across all the networks
(2) Sample a different network for each training instance

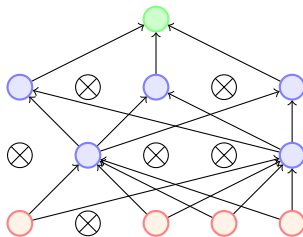
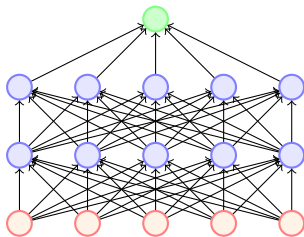


- Suppose a neural network has n nodes
- Using the dropout idea, each node can be retained or dropped
- For example, in the above case we drop 5 nodes to get a thinned network
- Given a total of n nodes, what are the total number of thinned networks that can be formed? 2^n
- Of course, this is prohibitively large and we cannot possibly train so many networks
- **Trick:** (1) Share the weights across all the networks
(2) Sample a different network for each training instance
- Let us see how?

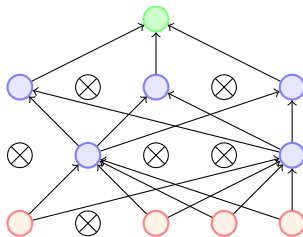
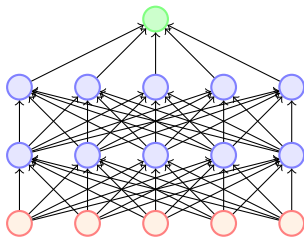




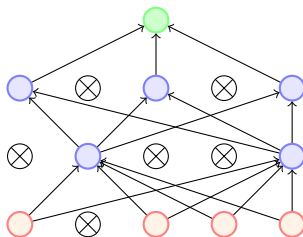
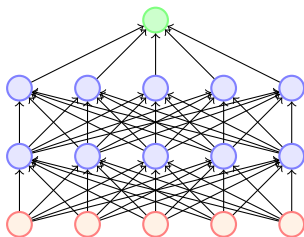
- We initialize all the parameters (weights) of the network and start training



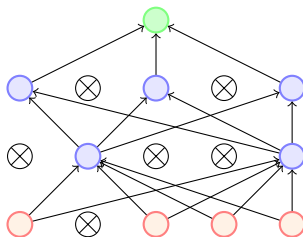
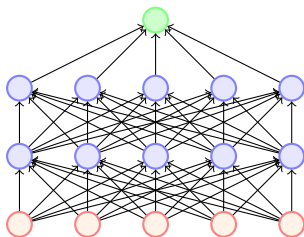
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network



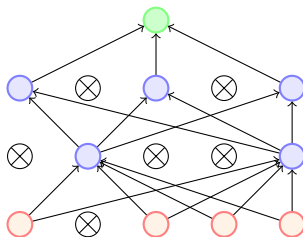
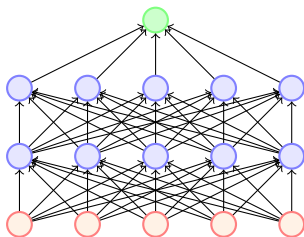
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network
- We compute the loss and backpropagate



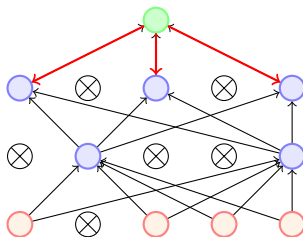
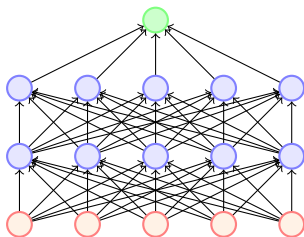
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network
- We compute the loss and backpropagate
- Which parameters will we update?



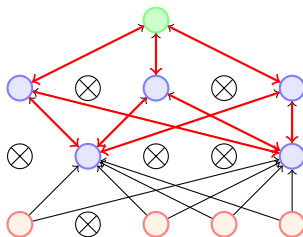
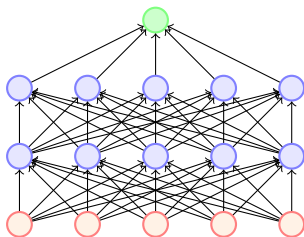
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network
- We compute the loss and backpropagate
- Which parameters will we update? Only those which are active



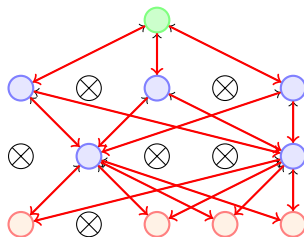
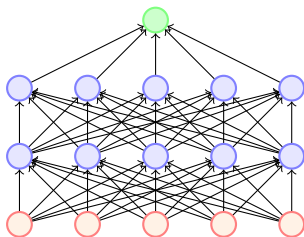
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network
- We compute the loss and backpropagate
- Which parameters will we update? Only those which are active



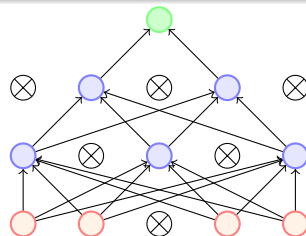
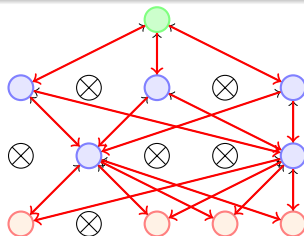
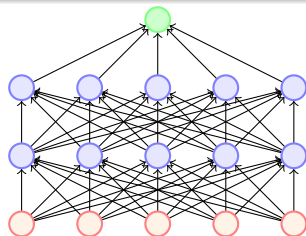
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network
- We compute the loss and backpropagate
- Which parameters will we update? Only those which are active



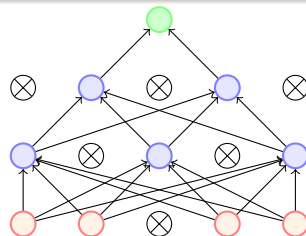
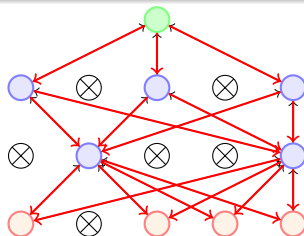
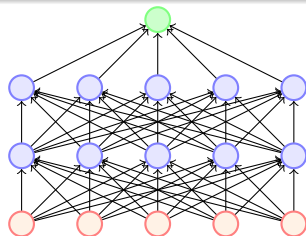
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network
- We compute the loss and backpropagate
- Which parameters will we update? Only those which are active



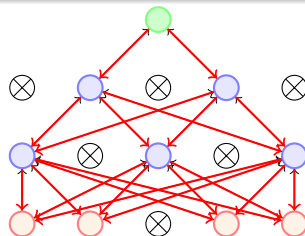
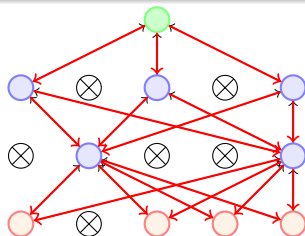
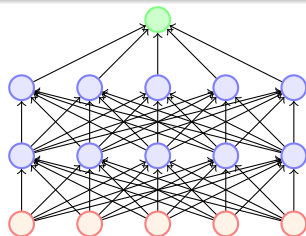
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network
- We compute the loss and backpropagate
- Which parameters will we update? Only those which are active



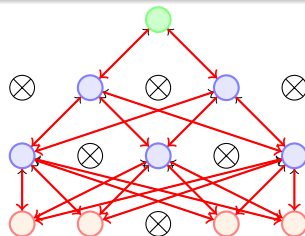
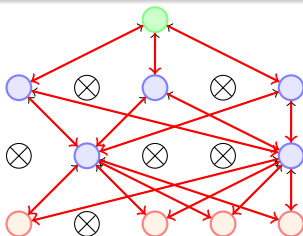
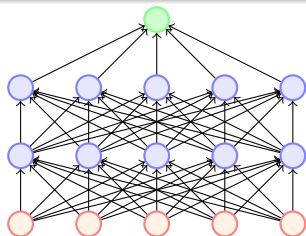
- For the second training instance (or mini-batch), we again apply dropout resulting in a different thinned network



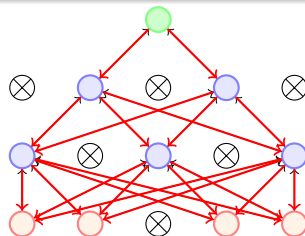
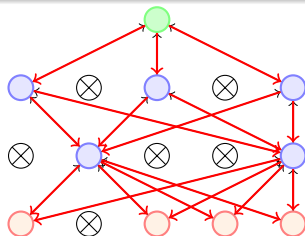
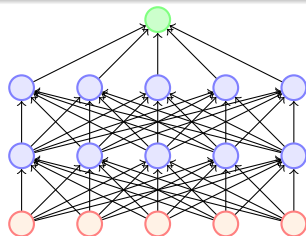
- For the second training instance (or mini-batch), we again apply dropout resulting in a different thinned network
- We again compute the loss and backpropagate to the active weights



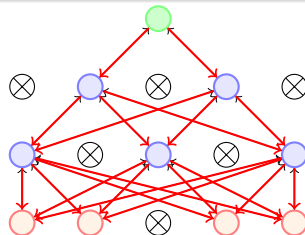
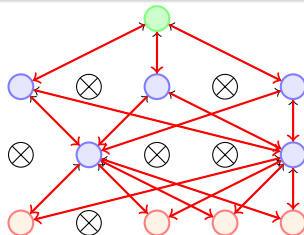
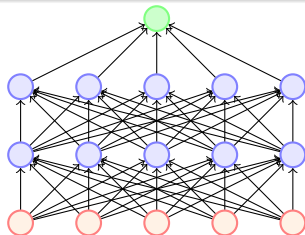
- For the second training instance (or mini-batch), we again apply dropout resulting in a different thinned network
- We again compute the loss and backpropagate to the active weights



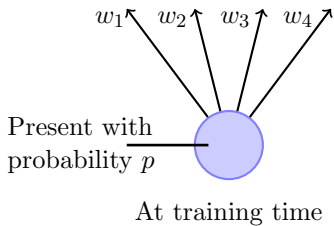
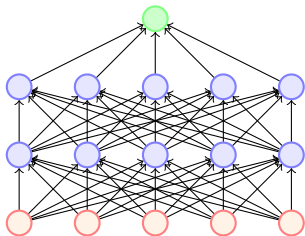
- For the second training instance (or mini-batch), we again apply dropout resulting in a different thinned network
- We again compute the loss and backpropagate to the active weights
- If the weight was active for both the training instances then it would have received two updates by now

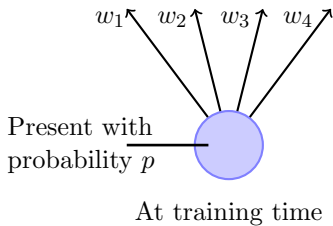
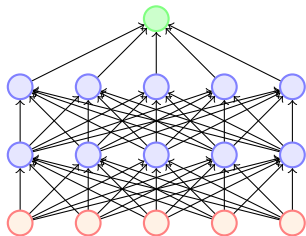


- For the second training instance (or mini-batch), we again apply dropout resulting in a different thinned network
- We again compute the loss and backpropagate to the active weights
- If the weight was active for both the training instances then it would have received two updates by now
- If the weight was active for only one of the training instances then it would have received only one updates by now

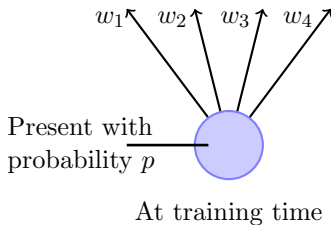
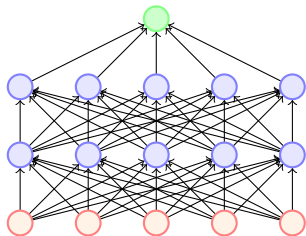


- For the second training instance (or mini-batch), we again apply dropout resulting in a different thinned network
- We again compute the loss and backpropagate to the active weights
- If the weight was active for both the training instances then it would have received two updates by now
- If the weight was active for only one of the training instances then it would have received only one updates by now
- Each thinned network gets trained rarely (or even never) but the parameter sharing ensures that no model has untrained or poorly trained parameters

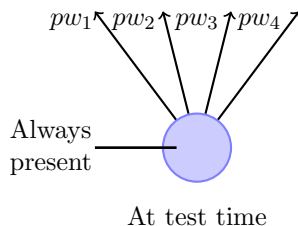
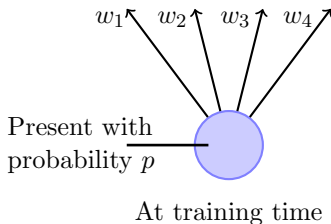
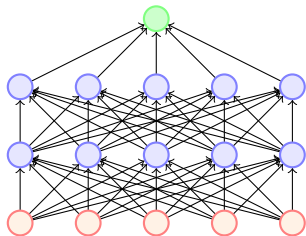




- What happens at test time?

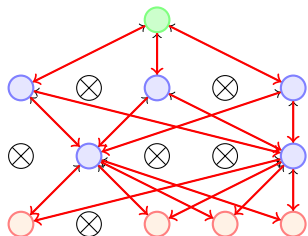


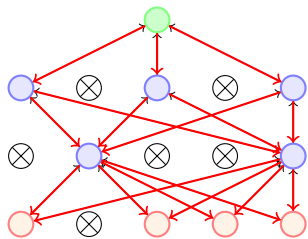
- What happens at test time?
- Impossible to aggregate the outputs of 2^n thinned networks



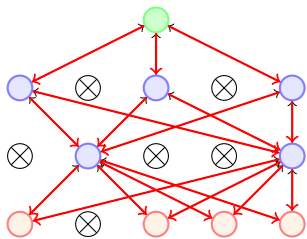
- What happens at test time?
- Impossible to aggregate the outputs of 2^n thinned networks
- Instead we use the full Neural Network and scale the output of each node by the fraction of times it was on during training

- Dropout essentially applies a masking noise to the hidden units

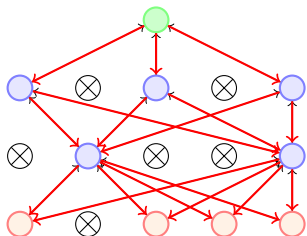




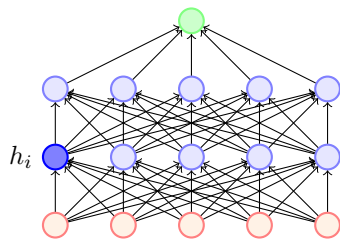
- Dropout essentially applies a masking noise to the hidden units
- Prevents hidden units from co-adapting



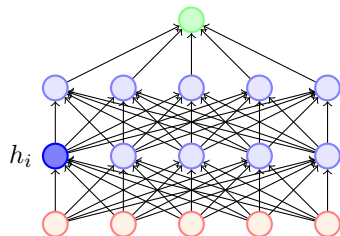
- Dropout essentially applies a masking noise to the hidden units
- Prevents hidden units from co-adapting
- Essentially a hidden unit cannot rely too much on other units as they may get dropped out any time

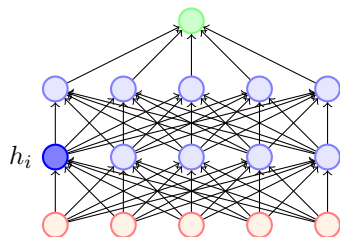


- Dropout essentially applies a masking noise to the hidden units
- Prevents hidden units from co-adapting
- Essentially a hidden unit cannot rely too much on other units as they may get dropped out any time
- Each hidden unit has to learn to be more robust to these random dropouts

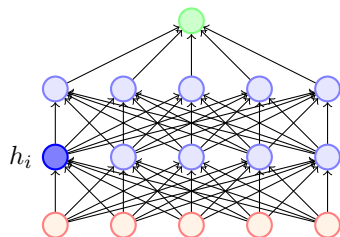


- Here is an example of how dropout helps in ensuring redundancy and robustness

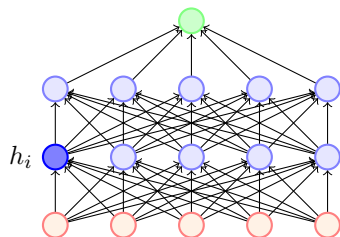




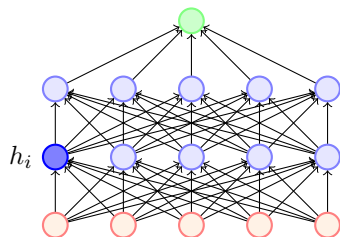
- Here is an example of how dropout helps in ensuring redundancy and robustness
- Suppose h_i learns to detect a face by firing on detecting a nose



- Here is an example of how dropout helps in ensuring redundancy and robustness
- Suppose h_i learns to detect a face by firing on detecting a nose
- Dropping h_i then corresponds to erasing the information that a nose exists



- Here is an example of how dropout helps in ensuring redundancy and robustness
- Suppose h_i learns to detect a face by firing on detecting a nose
- Dropping h_i then corresponds to erasing the information that a nose exists
- The model should then learn another h_i which redundantly encodes the presence of a nose



- Here is an example of how dropout helps in ensuring redundancy and robustness
- Suppose h_i learns to detect a face by firing on detecting a nose
- Dropping h_i then corresponds to erasing the information that a nose exists
- The model should then learn another h_i which redundantly encodes the presence of a nose
- Or the model should learn to detect the face using other features

Recap

- l_2 regularization
- Dataset augmentation
- Parameter Sharing and tying
- Adding Noise to the inputs
- Adding Noise to the outputs
- Early stopping
- Ensemble methods
- Dropout